



Escuela
Politécnica
Superior

Sistema robótico inteligente para la patrulla y detección de incidencias en estancias cerradas



Grado en Ingeniería Informática

Trabajo Fin de Grado

Autor:
Ginés Caballero Guijarro

Tutor:
Francisco A. Pujol López

Julio 2026



Universitat d'Alacant
Universidad de Alicante

Sistema robótico inteligente para la patrulla y detección de incidencias en estancias cerradas

Autor

Ginés Caballero Guijarro

Tutor

Francisco A. Pujol López

Tecnología Informática y Computación



Grado en Ingeniería Informática



Escuela
Politécnica
Superior



Universitat d'Alacant
Universidad de Alicante

ALICANTE, Julio 2026

Preámbulo

La vigilancia de espacios interiores cerrados (oficinas, naves, almacenes o viviendas) recae habitualmente en cámaras fijas y en la supervisión humana, dos recursos con cobertura limitada y elevado coste de atención continuada. La robótica móvil de servicio, unida a los avances recientes en visión por computador, abre la posibilidad de automatizar esta tarea mediante un agente capaz de recorrer el entorno de forma autónoma y de avisar únicamente cuando detecta una situación relevante. El presente Trabajo Fin de Grado se enmarca en este contexto y propone el diseño y la implementación de un sistema robótico de patrulla autónoma para interiores, construido sobre un robot TurtleBot 4 y Robot Operating System 2 (ROS 2), que cartografía el espacio, lo recorre siguiendo una ruta de cobertura, detecta la presencia de personas mediante una red neuronal de detección de objetos y notifica las incidencias a una plataforma centralizada con la que el usuario interactúa desde una aplicación móvil. El objetivo general es demostrar la viabilidad de una arquitectura desacoplada y orientada a eventos que integre la capa robótica, el procesamiento a bordo del robot y los servicios web en un único sistema funcional, extensible y seguro.

Agradecimientos

En primer lugar, quiero agradecer a mi tutor Paco y a Tamai por su orientación y su disponibilidad durante el desarrollo de este trabajo.

A mis padres y a mi pareja María, por confiar siempre en mí, incluso cuando yo no lo hacía.

A mis amigos Alejandro, Miguel, Nicolás, Juan, Raúl Luján y Raúl Ruiz, por el apoyo constante a lo largo de estos años y ser un pilar importantísimo en mi historia como ingeniero.

Finalmente, agradecer a la Escuela Politécnica Superior de la Universidad de Alicante la inestimable formación recibida durante el grado.

*A mi familia y pareja,
por su apoyo incondicional y por mantenerme siempre con los pies en la tierra.*

*“Solo podemos ver poco del futuro,
pero lo suficiente para darnos cuenta de que hay mucho por hacer.”*

Alan Turing, 1950

Resumen

La vigilancia de espacios interiores cerrados depende en gran medida de cámaras fijas y de supervisión humana, soluciones de cobertura limitada y atención costosa. Este trabajo aborda la automatización de dicha tarea mediante un robot móvil capaz de patrullar de forma autónoma y de alertar solo ante eventos relevantes. Para ello se ha diseñado e implementado un sistema completo sobre un robot TurtleBot 4 y ROS 2, organizado como un monorepo con cuatro bloques: la capa robótica, una infraestructura de datos en contenedores, un servicio de backend y una aplicación de usuario. El robot construye el mapa del entorno mediante Simultaneous Localization and Mapping (SLAM), genera una ruta de cobertura a partir de dicho mapa y la recorre con la pila de navegación ROS 2 Navigation Stack (Nav2), mientras un nodo de percepción basado en You Only Look Once (YOLO) detecta personas y registra las incidencias. La comunicación entre el robot y el servidor se resuelve con una arquitectura orientada a eventos basada en un bus de mensajería, lo que desacopla el hardware del plano de control y evita la dependencia de una conexión permanente entre ambos extremos. El backend, desarrollado con FastAPI, consume estos eventos, persiste los datos y expone una Application Programming Interface (API) Representational State Transfer (REST) consumida por una aplicación Flutter desde la que el usuario controla la patrulla y revisa los incidentes. Los resultados presentados son de carácter preliminar y se centran en la validación funcional de la arquitectura; no obstante, el sistema sienta una base sólida y extensible para el desarrollo de soluciones de vigilancia robótica autónoma.

Palabras clave: robótica móvil, ROS 2, navegación autónoma, visión por computador, arquitectura orientada a eventos, vigilancia.

Índice general

Preámbulo	v
Agradecimientos	vii
Resumen	xiii
1 Introducción	1
1.1 Importancia y actualidad de la temática	1
1.2 Planteamiento del problema	1
1.3 Propósito y finalidad del trabajo	2
1.4 Estructura de la memoria	2
2 Marco teórico	3
2.1 Robótica móvil autónoma y ROS 2	3
2.2 Localización y mapeado simultáneos (SLAM)	4
2.3 Navegación autónoma y planificación de rutas de cobertura	4
2.4 Visión por computador y detección de objetos	5
2.5 Arquitecturas orientadas a eventos	6
2.6 Trabajos relacionados	6
2.7 Síntesis y posicionamiento	7
3 Objetivos	9
3.1 Objetivo general	9
3.2 Objetivos específicos	9
4 Metodología	11
4.1 Metodología de desarrollo	11
4.2 Herramientas, frameworks y entornos	11
4.2.1 Plataforma robótica y navegación	12
4.2.2 Percepción visual	12
4.2.3 Mensajería orientada a eventos	13
4.2.4 Backend y servicios web	13
4.2.5 Persistencia, almacenamiento y visualización	13
4.2.6 Aplicación de usuario	13
4.2.7 Infraestructura y despliegue	14
4.2.8 Entorno de desarrollo y herramientas de apoyo	14
4.2.9 Bibliotecas y dependencias principales	14
4.3 Criterios y métricas de evaluación	16

5	Desarrollo	17
5.1	Vista general y flujos de datos	17
5.1.1	Cliente → servidor → robot: control de la patrulla	18
5.1.2	Robot → servidor → cliente: telemetría, incidencias y estado	18
5.1.3	Autenticación y arranque: TOFU y JWT	19
5.2	Arquitectura e infraestructura	19
5.3	Backend	21
5.3.1	Modelo de datos	21
5.3.2	API REST	21
5.3.3	Consumidores y productor de eventos	23
5.3.4	Seguridad	24
5.4	Capa robótica	25
5.4.1	Orquestador: patrol_nav	26
5.4.2	Detección: detect_incident	30
5.4.3	Telemetría: serve_telemetry	31
5.4.4	Gestión de secretos: secret_helper	32
5.5	Aplicación de usuario	34
5.5.1	Arquitectura de la aplicación	34
5.5.2	Recorrido por las pantallas	35
6	Resultados	41
6.1	Entorno y escenario de pruebas	41
6.2	Validación funcional	41
6.3	Resultados de simulación	42
6.3.1	Cartografía y ruta de patrulla	42
6.3.2	Detección de incidencias	44
6.3.3	Validación del flujo de extremo a extremo	46
6.4	Pruebas en el robot real	47
6.4.1	Entorno y metodología	47
6.4.2	Sincronización temporal: los tres relojes	48
6.4.3	Subsistemas validados en hardware real	48
6.4.4	Navegación y patrulla: validación parcial	50
6.4.5	Detección de incidencias: no evaluada por un fallo de la cámara	52
6.4.6	Síntesis de la campaña real	53
6.5	Discusión	54
7	Conclusiones	55
7.1	Grado de consecución de los objetivos	55
7.2	Aportaciones	55
7.3	Limitaciones	56
7.4	Trabajo futuro	57
7.5	Valoración personal	58
	Bibliografía	59
	Lista de Acrónimos y Abreviaturas	61

Índice de figuras

5.1	Arquitectura del sistema por capas	17
5.2	Máquina de estados de patrol_nav	28
5.3	Panel de Grafana que visualiza la telemetría del robot en tiempo real. Fuente: elaboración propia.	32
5.4	Arranque inicial y autenticación de secret_helper	33
5.5	Obtención y uso del JWT por el resto de nodos	34
5.6	Menú de navegación de la aplicación	35
5.7	Pantalla de acceso	37
5.8	Pantalla de inicio	38
5.9	Pantalla de gestión de robots	38
5.10	Pantalla de incidencias	39
5.11	Pantalla de perfil	39
5.12	Panel de administración	40
6.1	Mapa y ruta de patrulla en simulación	43
6.2	Detección de una persona en simulación	45
6.3	Autenticación TOFU en el robot real	49
6.4	Telemetría del robot real en Grafana	50
6.5	Exploración en curso en el robot real	51

Índice de tablas

4.1	Principales tecnologías empleadas por capa del sistema. Fuente: elaboración propia.	12
4.2	Bibliotecas y dependencias principales, su función y el componente en el que se emplean. Fuente: elaboración propia.	15
5.1	Resumen de los flujos de datos del sistema. Fuente: elaboración propia. . .	19
5.2	Servicios de infraestructura y puertos publicados. Fuente: elaboración propia.	20
5.3	Endpoints de la API REST	22
5.4	Nodos ROS 2 propios del sistema. Fuente: elaboración propia.	26
5.5	Estados de la máquina de estados de patrol_nav. Fuente: elaboración propia.	27
5.6	Campos del mensaje de telemetría, con su tipo y la suscripción de origen. Fuente: elaboración propia.	32
5.7	Pantallas de la aplicación, su ruta y los endpoints que consumen. Fuente: elaboración propia.	36
6.1	Resumen de la validación funcional realizada en simulación. Fuente: elaboración propia.	42
6.2	Métricas de cartografía y navegación en simulación. Fuente: elaboración propia.	43
6.3	Métricas de detección sobre el escenario controlado en simulación. Fuente: elaboración propia.	45
6.4	Métricas del flujo de extremo a extremo medibles en simulación, independientes del factor de tiempo real. Fuente: elaboración propia.	46
6.5	Degradación de la entrega del escáner láser (/scan) en la máquina virtual, medida en reposo frente a bajo carga de la pila de navegación (n. d.: no medido de forma fiable bajo carga). Fuente: elaboración propia.	51
6.6	Estado de validación de los subsistemas sobre el robot real. Fuente: elaboración propia.	53

Índice de Códigos

5.1	YAML	20
5.2	Python	24
5.3	Python	25
5.4	Python	29
5.5	Python	29
5.6	Python	30
5.7	JSON	31
5.8	Dart	35
7.1	YAML	57

1 Introducción

La protección y la supervisión de espacios interiores son una necesidad constante en entornos tan diversos como oficinas, naves industriales, almacenes o viviendas. Este capítulo enmarca el trabajo en ese ámbito: se argumenta la importancia y la actualidad del problema, se delimita el reto técnico que se aborda y se enuncia el propósito del presente Trabajo Fin de Grado, para terminar describiendo la organización de la memoria.

1.1 Importancia y actualidad de la temática

La vigilancia de espacios interiores cerrados se apoya tradicionalmente en redes de cámaras fijas y en la supervisión por parte de personal de seguridad. Ambos recursos presentan limitaciones conocidas: las cámaras fijas solo cubren los ángulos en los que se han instalado y dejan zonas ciegas, y la atención humana sostenida sobre múltiples monitores resulta costosa y propensa a la fatiga. Como consecuencia, una parte de los eventos relevantes puede pasar inadvertida o detectarse con retraso, justo cuando una respuesta temprana es más valiosa.

El interés por automatizar esta tarea es especialmente vigente hoy en día. En los últimos años, la robótica móvil ha alcanzado un grado de madurez que permite delegar parte de la vigilancia en plataformas autónomas: la consolidación de ROS 2 como middleware estándar para el desarrollo de aplicaciones robóticas, con despliegues reales documentados (Macenski et al., 2022), junto con la aparición de pilas de navegación robustas (Macenski et al., 2020) y de hardware asequible, ha rebajado de forma notable la barrera de entrada. De manera complementaria, los detectores de objetos en tiempo real basados en redes neuronales convolucionales (Redmon et al., 2016), hoy disponibles en implementaciones eficientes y de uso extendido (Jocher et al., 2023), hacen viable dotar a un robot de una capacidad de percepción suficiente para identificar situaciones de interés mientras se desplaza.

La convergencia de ambos avances habilita un modelo de vigilancia activa, en el que un agente móvil recorre el espacio de forma autónoma y reclama la atención del operador únicamente ante los eventos que detecta. Frente al esquema pasivo de la cámara fija, este enfoque amplía la cobertura efectiva con un único dispositivo en movimiento y filtra la información que llega al usuario. Es en esta ventana de oportunidad donde se sitúa el presente trabajo.

1.2 Planteamiento del problema

Construir un robot de patrulla autónoma para interiores no constituye un problema único, sino la integración coordinada de varios subproblemas. El robot debe cartografiar un entorno inicialmente desconocido, planificar una ruta que cubra el espacio de forma razonable (Galceran & Carreras, 2013), navegar evitando obstáculos, percibir la presencia de personas y comunicar las incidencias a un sistema con el que el usuario pueda interactuar. Cada una de estas capacidades descansa en tecnologías distintas (navegación, percepción, mensajería, servicios web y aplicaciones de usuario) que rara vez comparten el mismo plano de ejecución.

El reto principal, por tanto, no es resolver cada subproblema de manera aislada, sino integrarlos en un sistema coherente que sea a la vez funcional, seguro y mantenible. Un acoplamiento excesivo entre el robot y el servidor, en el que cada extremo depende de la disponibilidad inmediata del otro, penalizaría la escalabilidad y la robustez del sistema y elevaría el consumo de recursos de una plataforma con autonomía energética limitada. De ahí la necesidad de crear una arquitectura que desacople la capa física del robot del plano de control y de los servicios de usuario.

1.3 Propósito y finalidad del trabajo

El propósito de este Trabajo Fin de Grado es diseñar e implementar un sistema robótico de patrulla autónoma para estancias cerradas que dé respuesta al problema anterior, construido sobre un robot TurtleBot 4 y ROS 2. El sistema cartografía el entorno mediante SLAM, deriva del mapa resultante una ruta de cobertura y la recorre con la pila de navegación Nav2. Durante el recorrido, un módulo de percepción basado en YOLO detecta la presencia de personas, graba un clip del evento y lo registra como incidencia.

La finalidad última no es solo que el robot patrulle, sino demostrar la viabilidad de una arquitectura distribuida y orientada a eventos que integre la capa robótica, el procesamiento a bordo y los servicios web en un único sistema funcional, extensible y seguro. La comunicación entre el robot y el resto del sistema se resuelve mediante un bus de mensajería (Kreps et al., 2011) que desacopla el hardware del backend, de modo que el robot solo transmite información cuando se produce un evento relevante. Un servidor desarrollado con FastAPI consume estos eventos, persiste la información y expone una API REST que una aplicación Flutter emplea para que el usuario controle la patrulla y revise el historial de incidencias. El proyecto se organiza en cuatro bloques diferenciados (capa robótica, infraestructura de datos, backend y aplicación de usuario) desplegados mediante contenedores.

1.4 Estructura de la memoria

El resto del documento se organiza como sigue. El Capítulo 2 revisa los fundamentos teóricos y el estado del arte en navegación autónoma, percepción visual y arquitecturas orientadas a eventos. El Capítulo 3 formula el objetivo general y los objetivos específicos del trabajo. El Capítulo 4 describe la metodología de desarrollo, las herramientas empleadas y los criterios de evaluación previstos. El Capítulo 5 constituye el núcleo técnico de la memoria y detalla la arquitectura del sistema y la implementación de cada uno de sus subsistemas. El Capítulo 6 presenta los resultados obtenidos y su análisis. Finalmente, el Capítulo 7 recoge las conclusiones, las limitaciones detectadas y las líneas de trabajo futuro.

2 Marco teórico

Este capítulo presenta los fundamentos teóricos sobre los que se sustenta el trabajo y revisa el estado del arte de las disciplinas implicadas. Se abordan, en primer lugar, el middleware robótico y el problema de la localización y el mapeado; a continuación, la navegación autónoma y la planificación de rutas de cobertura; después, la detección de objetos mediante visión por computador; y, por último, las arquitecturas orientadas a eventos. El capítulo concluye situando la propuesta respecto a los trabajos relacionados.

2.1 Robótica móvil autónoma y ROS 2

ROS 2 es un middleware de código abierto para el desarrollo de aplicaciones robóticas que proporciona un modelo de comunicación entre procesos, herramientas de construcción y un ecosistema amplio de paquetes reutilizables. Su primera generación se introdujo con el objetivo de favorecer la reutilización de software en investigación robótica (Quigley et al., 2009). Sin embargo, su diseño dependía de un nodo maestro central que coordinaba el descubrimiento entre procesos: todos los nodos debían registrarse en él para poder localizarse entre sí, por lo que constituía un punto único de fallo. Si el maestro dejaba de funcionar, los nodos no podían descubrir a sus interlocutores y el sistema en su conjunto quedaba inoperativo. Esta dependencia, junto con la ausencia de garantías de tiempo real, limitaba su uso en entornos de producción y en sistemas con varios robots. La segunda generación, ROS 2, rediseña la arquitectura para superar esas carencias: suprime el nodo maestro y delega el descubrimiento y el transporte de mensajes en el estándar Data Distribution Service (DDS), de modo que los nodos se descubren directamente entre sí, sin coordinador central. Así desaparece el punto único de fallo y se obtiene una comunicación descentralizada, con control de la calidad de servicio de los enlaces y mejor soporte para despliegues distribuidos (Macenski et al., 2022).

El modelo de programación de ROS 2 se organiza en torno a *nodos*, procesos independientes que encapsulan una funcionalidad concreta y se comunican mediante el paso de mensajes. Los nodos intercambian flujos de datos a través de *topics* con un patrón de publicación-suscripción, invocan operaciones puntuales mediante *servicios* de tipo petición-respuesta y gestionan tareas de larga duración, con realimentación y posibilidad de cancelación, mediante *acciones*. Sobre el transporte DDS, cada enlace puede ajustar parámetros de calidad de servicio, como la fiabilidad de la entrega o la persistencia de los últimos valores publicados, lo que permite adaptar la comunicación tanto a sensores de alta frecuencia como a eventos esporádicos.

Más allá del modelo de comunicación, ROS 2 ofrece un sistema de construcción y empaquetado, un mecanismo de parámetros para configurar los nodos en tiempo de ejecución y un sistema de lanzamiento que describe cómo arrancar y orquestar conjuntos de nodos. Esta separación en componentes débilmente acoplados favorece la modularidad, permite reutilizar paquetes mantenidos por la comunidad y hace posible sustituir una pieza sin alterar el resto del sistema. El sistema desarrollado adopta ese principio de forma explícita; el detalle de cómo

se reparten las capacidades del robot en nodos especializados se aborda en el Capítulo 5.

2.2 Localización y mapeado simultáneos (SLAM)

Para navegar de forma autónoma, un robot necesita conocer tanto el mapa del entorno como su propia posición dentro de él. Cuando ninguno de los dos es conocido de antemano, el problema se formula como localización y mapeado simultáneos, o SLAM: la construcción incremental de un mapa de un entorno desconocido a la vez que se estima la trayectoria del robot (Durrant-Whyte & Bailey, 2006). Se trata de un problema fundamental en robótica móvil porque ambas estimaciones son interdependientes, ya que un buen mapa requiere conocer la posición y una buena posición requiere conocer el mapa, y porque los errores de odometría se acumulan a lo largo del recorrido y producen deriva si no se corrigen.

Resolver el SLAM implica abordar a la vez dos tareas acopladas. La primera consiste en interpretar las medidas de los sensores y resolver la asociación de datos, esto es, decidir qué observaciones corresponden a un mismo elemento real del entorno. La segunda estima, a partir de esas asociaciones, la trayectoria del robot y el mapa más coherentes con todas las observaciones acumuladas. En este proceso, la detección de cierres de bucle (reconocer que el robot ha vuelto a un lugar ya visitado) resulta determinante para corregir la deriva acumulada por la odometría. Históricamente, las primeras formulaciones se apoyaban en filtros recursivos, como el filtro de Kalman extendido; los enfoques predominantes en la actualidad plantean el problema como la optimización de un grafo de poses, más preciso y escalable para trayectorias largas.

Según el sensor empleado, se distingue entre el SLAM visual, basado en cámaras, y el SLAM láser, basado en un sensor Light Detection and Ranging (LiDAR) que mide distancias mediante haces de luz. En el ámbito de ROS 2, una de las soluciones más extendidas para interiores es *slam_toolbox*, que implementa SLAM láser basado en grafos de poses y permite tanto el mapeado en línea como la posterior localización sobre el mapa generado, además de guardar y recargar mapas previamente construidos (Macenski & Jambrecic, 2021). El presente trabajo emplea este enfoque para que el robot construya el mapa de la estancia durante una fase inicial de exploración.

2.3 Navegación autónoma y planificación de rutas de cobertura

Una vez disponible el mapa, la navegación autónoma consiste en planificar y ejecutar trayectorias seguras hacia los objetivos deseados evitando obstáculos. En ROS 2, esta funcionalidad la proporciona Nav2, una pila de navegación que integra un planificador global, un controlador local y un conjunto de comportamientos de recuperación, coordinados mediante árboles de comportamiento (*behavior trees*) (Macenski et al., 2020). El planificador global calcula una ruta hasta el objetivo sobre una representación del entorno conocida como mapa de costes (*costmap*), que combina los obstáculos del mapa con una zona de inflado de seguridad alrededor de ellos; el controlador local genera las órdenes de velocidad que siguen esa ruta mientras esquiva obstáculos imprevistos; y los comportamientos de recuperación son maniobras predefinidas que se activan cuando la navegación falla o el robot queda bloqueado. Entre ellas figuran limpiar los mapas de costes para descartar obstáculos transitorios o espurios, girar sobre sí mismo para volver a percibir el entorno, retroceder o esperar un instante.

El árbol de comportamiento las ejecuta de forma ordenada cuando un intento de navegación no progresa y, tras ellas, reintenta alcanzar el objetivo, de modo que el robot se desatasca sin intervención humana.

El empleo de árboles de comportamiento, en lugar de una máquina de estados fija, dota a la navegación de una lógica de decisión modular y fácil de modificar, en la que pueden añadirse o reordenarse comportamientos sin reescribir el sistema. Nav2 expone, además, interfaces de alto nivel para enviar al robot a una pose objetivo o a través de una secuencia ordenada de poses, una capacidad especialmente adecuada para las tareas de patrulla, en las que el robot debe recorrer una lista de puntos predefinidos.

El recorrido sistemático de un espacio se corresponde con el problema de planificación de rutas de cobertura (*coverage path planning*), que busca una trayectoria que haga pasar al robot por todas las regiones accesibles del entorno (Galceran & Carreras, 2013). Existen múltiples estrategias, desde las descomposiciones celulares, que dividen el espacio en regiones simples recorridas en barridos de ida y vuelta (*boustrophedon*), hasta los métodos basados en rejilla o en árboles de expansión, y la elección entre ellas responde al compromiso entre completitud de la cobertura, longitud de la ruta y coste de cálculo. De entre estas estrategias, este trabajo opta por una basada en rejilla por su sencillez; su diseño concreto y su justificación se detallan en el Capítulo 5.

2.4 Visión por computador y detección de objetos

La detección de objetos consiste en localizar y clasificar las instancias de interés presentes en una imagen, devolviendo para cada una un recuadro delimitador (*bounding box*) y una categoría con su grado de confianza. Los métodos modernos se apoyan en redes neuronales convolucionales (CNN), capaces de aprender directamente de los datos las características visuales relevantes. Tradicionalmente se distinguen dos familias: los detectores de dos etapas, como Region-based Convolutional Neural Network (R-CNN), que primero proponen regiones candidatas y después las clasifican, con alta precisión pero mayor coste computacional; y los detectores de una sola etapa, que resuelven la localización y la clasificación en una única pasada sobre la imagen.

La familia YOLO pertenece a esta segunda categoría y supuso un avance decisivo al plantear la detección como un único problema de regresión sobre la imagen completa, dividida en una rejilla en la que cada celda predice recuadros y probabilidades de clase, lo que permitió alcanzar velocidades adecuadas para el procesamiento en tiempo real (Redmon et al., 2016). Un paso posterior de supresión (*non-maximum suppression*) descarta las detecciones redundantes que se solapan sobre un mismo objeto. A lo largo de sucesivas versiones, YOLO ha mejorado la precisión y la eficiencia y ha reducido el tamaño de los modelos.

Para su despliegue en plataformas con recursos limitados resulta habitual emplear las variantes más ligeras, que ofrecen un compromiso favorable entre precisión y coste computacional, frente a las variantes mayores que requieren una GPU potente. En este trabajo se recurre a una de estas variantes ligeras, en la implementación de Ultralytics (Jocher et al., 2023), ejecutada en el propio dispositivo; el modelo concreto y la justificación de la elección se detallan en el Capítulo 5. Este procesamiento en el propio robot (*edge computing*) evita transmitir vídeo de forma continua a un servidor, reduce la carga de red y favorece la privacidad, al analizar las imágenes en el propio dispositivo.

2.5 Arquitecturas orientadas a eventos

En los sistemas distribuidos, una arquitectura orientada a eventos organiza la comunicación en torno a la producción y el consumo de eventos a través de un intermediario de mensajería (*broker*), en lugar de mediante llamadas directas entre componentes. Un evento es un registro inmutable de algo que ha sucedido, por ejemplo una lectura de telemetría o la detección de una incidencia. Este patrón desacopla a los productores de los consumidores: ambos desconocen su existencia mutua y se comunican únicamente a través del bus, de modo que añadir un nuevo consumidor no obliga a modificar al productor.

Una de las plataformas de referencia para este propósito es Apache Kafka. Utilizado por empresas como Netflix, Spotify o Uber, Kafka es un sistema de mensajería distribuido y persistente diseñado para procesar grandes volúmenes de flujos de datos con baja latencia (Kreps et al., 2011). Kafka organiza los eventos en *topics*, que a su vez se reparten en particiones para escalar de forma horizontal, y conserva los mensajes en un registro durante un periodo configurable. Esta persistencia permite que distintos consumidores lean el mismo flujo a su propio ritmo, e incluso reprocesen eventos pasados, una propiedad difícil de obtener con una comunicación síncrona y efímera.

Este enfoque encaja bien con un robot de patrulla: al publicar la telemetría y las incidencias como eventos, el robot solo transmite cuando ocurre algo relevante y no necesita mantener conexiones síncronas con el servidor, lo que reduce el consumo y mejora la tolerancia a desconexiones temporales. El modo en que el sistema desarrollado articula este patrón, y su contraste con un modelo basado únicamente en peticiones REST, se desarrolla en el Capítulo 5.

2.6 Trabajos relacionados

La vigilancia mediante robots móviles autónomos cuenta con una trayectoria consolidada en la literatura. Ya en propuestas tempranas se integraban el mapeado, la localización y la navegación autónoma con tareas propias de la vigilancia, como la detección de objetos abandonados o el seguimiento de personas (Di Paola et al., 2010). Estos sistemas demostraron la viabilidad de delegar la inspección de un entorno interior en una plataforma autónoma y sentaron las bases sobre las que se apoyan los trabajos posteriores.

Las propuestas más recientes incorporan el aprendizaje profundo para la percepción y se construyen sobre ROS 2. Existen robots de patrulla que combinan SLAM láser con cámaras RGB-D y emplean detectores de la familia YOLO para generar mapas semánticos y evitar obstáculos durante el recorrido (Chiang & Huang, 2026). En la línea de la detección de intrusos, otras propuestas despliegan flotas de robots coordinados bajo ROS 2, equipados con sensores LiDAR bidimensionales, que agregan sus detecciones en un nodo central (Islam et al., 2020). Estos trabajos confirman la idoneidad de la combinación de ROS 2, SLAM y detección basada en redes neuronales, que también adopta el presente trabajo.

No obstante, la mayoría de estas soluciones concentran su esfuerzo en la propia plataforma robótica, es decir, en la navegación y la percepción, y tratan de forma secundaria, cuando no omiten, la integración con servicios web, la persistencia y la gestión de las incidencias y la interacción con el usuario final a través de una aplicación. Muchas son, además, sistemas cerrados o difíciles de reproducir.

La propuesta de este trabajo se diferencia por abordar el sistema de forma integral y con tecnologías abiertas: cubre desde la cartografía y la patrulla autónoma hasta la notificación de incidencias y su consulta desde una aplicación móvil, articulando todo ello mediante una arquitectura orientada a eventos. Esta visión de extremo a extremo, más que la novedad de cada componente por separado, constituye la aportación principal.

2.7 Síntesis y posicionamiento

A lo largo del capítulo se han revisado tanto los fundamentos que sustentan el trabajo, ROS 2 como middleware, el SLAM para el mapeado, Nav2 para la navegación, la detección de objetos con YOLO para la percepción y las arquitecturas orientadas a eventos para la comunicación, como los trabajos previos que abordan problemas afines. Cada una de esas tecnologías constituye una pieza madura y consolidada en su ámbito, y los sistemas de vigilancia robótica existentes confirman su aplicabilidad conjunta. Frente a esos trabajos, que suelen detenerse en la plataforma robótica, el valor del presente reside en integrar todas esas piezas, junto con el servidor y la aplicación de usuario, en un único sistema funcional de extremo a extremo, cuyo diseño y construcción se detallan en el Capítulo 5, tras formular los objetivos en el Capítulo 3.

3 Objetivos

A partir del problema planteado en el Capítulo 1 y del marco revisado en el Capítulo 2, este capítulo formula el objetivo general del trabajo y lo desglosa en una serie de objetivos específicos verificables que guían el desarrollo descrito en los capítulos posteriores.

3.1 Objetivo general

El objetivo general del trabajo es diseñar e implementar un sistema robótico de patrulla autónoma para estancias cerradas que cartografie el entorno, lo recorra de forma autónoma, detecte la presencia de personas como incidencia y la notifique a una plataforma centralizada con la que el usuario interactúe, todo ello articulado mediante una arquitectura orientada a eventos que desacople la capa robótica del plano de control.

3.2 Objetivos específicos

Para alcanzar el objetivo general se definen los siguientes objetivos específicos:

1. **OE1. Cartografía autónoma del entorno.** Dotar al robot de la capacidad de explorar de forma autónoma una estancia desconocida, construir su mapa mediante SLAM y almacenarlo para su uso posterior.
2. **OE2. Generación y ejecución de la ruta de patrulla.** Derivar del mapa una ruta de cobertura del espacio libre y recorrerla de manera cíclica con la pila de navegación Nav2, evitando obstáculos.
3. **OE3. Orquestación del ciclo de patrulla.** Diseñar un orquestador, en forma de una máquina de estados finita (FSM), que coordine las fases del ciclo operativo del robot (exploración y mapeado, planificación de la ruta, patrulla y retorno al punto de carga), de modo que el robot encadene de forma autónoma las capacidades anteriores.
4. **OE4. Percepción y detección de incidencias en el entorno.** Integrar un módulo de visión por computador basado en YOLO que detecte personas durante la patrulla, grabe un clip del evento y lo registre como incidencia.
5. **OE5. Arquitectura orientada a eventos.** Diseñar el sistema de comunicación basado en un bus de mensajería que desacople el robot del backend y canalice la telemetría, las incidencias y los comandos de control.
6. **OE6. Backend y persistencia de datos.** Desarrollar un servicio de backend que consuma los eventos del robot, exponga una API REST y persista la información en los almacenes adecuados, incluyendo el vídeo de las incidencias y las series temporales de telemetría.

7. **OE7. Seguridad y autenticación.** Implementar un mecanismo de autenticación tanto para el robot, mediante un esquema de confianza en el primer uso (Trust On First Use (TOFU)) y tokens JSON Web Token (JWT), como para los usuarios, con control de acceso basado en roles.
8. **OE8. Aplicación de usuario.** Desarrollar una aplicación multiplataforma con Flutter desde la que el usuario controle la patrulla del robot y consulte el historial de incidencias y la telemetría.
9. **OE9. Integración y validación funcional.** Integrar todos los subsistemas en una solución desplegable mediante contenedores y comprobar su funcionamiento conjunto.

El grado de consecución de cada uno de estos objetivos se valora en el Capítulo 7, a la luz de los resultados presentados en el Capítulo 6.

4 Metodología

Este capítulo describe el enfoque metodológico seguido para alcanzar los objetivos planteados, las herramientas y entornos empleados, y los criterios de evaluación previstos para valorar el sistema. La descripción detallada del diseño y la implementación se reserva para el Capítulo 5.

4.1 Metodología de desarrollo

El trabajo se ha abordado mediante un enfoque de desarrollo **iterativo e incremental**, organizado en sprints sucesivos cuya gestión se ha apoyado en un tablero *Kanban*. En este tablero, las tareas avanzan por columnas según su estado, lo que ha permitido mantener una visión clara del progreso y priorizar el trabajo pendiente en cada iteración. En lugar de construir la solución completa de una sola vez, se ha avanzado por incrementos funcionales, validando cada parte antes de integrarla con el resto.

El orden de construcción ha seguido la cadena de dependencias del sistema. En primer lugar se levantó la infraestructura de datos (las bases de datos, el bus de mensajería y los servicios de almacenamiento y visualización), por ser el soporte sobre el que se apoya todo lo demás. A continuación se desarrolló el backend, que centraliza la lógica de negocio y la API. Después se abordó la capa robótica (**robot_ws**), que produce los datos y consume los servicios del backend. Y, por último, se implementó la aplicación de usuario, que también consume la API del backend.

La integración entre el robot y el backend se realizó funcionalidad a funcionalidad, siguiendo siempre el mismo patrón: primero se implementaba en el backend el *endpoint* o la lógica necesaria para una capacidad concreta, y a continuación se desarrollaba la parte del robot que se conecta a ella. Este encadenamiento, repetido para cada funcionalidad (telemetría, incidencias, comandos de control y autenticación), garantizaba que cada extremo dispusiera ya de su contraparte y que cada incremento fuera verificable de principio a fin.

Finalmente, todo el código se ha organizado como un **repositorio monolítico**, es decir, un único repositorio que agrupa todos los componentes del sistema, lo que facilita la coherencia entre las distintas partes y simplifica el despliegue conjunto. El repositorio se estructura en cuatro bloques principales: la capa robótica (**robot_ws**), el backend (**backend**), la aplicación de usuario (**flutter_frontend**) y un conjunto de *scripts* de automatización del despliegue. La arquitectura concreta de cada bloque se detalla en el Capítulo 5.

4.2 Herramientas, frameworks y entornos

El sistema se apoya en un conjunto de tecnologías de código abierto, seleccionadas por su madurez y por su integración con el ecosistema de ROS 2. La Tabla 4.1 resume las principales herramientas empleadas en cada capa del sistema.

Tabla 4.1: Principales tecnologías empleadas por capa del sistema. Fuente: elaboración propia.

Capa	Tecnologías
Plataforma robótica	TurtleBot 4, ROS 2 Humble
Navegación y mapeado	Nav2, slam_toolbox, explore_lite
Percepción	YOLOv8n (Ultralytics), OpenCV
Mensajería	Apache Kafka (cliente confluent-kafka)
Backend	FastAPI, SQLAlchemy, Uvicorn
Persistencia	PostgreSQL, InfluxDB, MinIO
Visualización	Grafana
Aplicación de usuario	Flutter
Infraestructura	Docker, Docker Compose

A continuación se detalla el papel de cada tecnología y la justificación de su elección, agrupadas por capa.

4.2.1 Plataforma robótica y navegación

El robot empleado es el TurtleBot 4, una plataforma de robótica móvil de referencia compuesta por una base *Create 3*, un computador de a bordo (Raspberry Pi 4), un sensor LiDAR y una cámara OAK-D. Se ha elegido por ser una plataforma estándar, asequible y con soporte oficial para ROS 2, lo que evita desviar el esfuerzo del trabajo hacia la integración de hardware a bajo nivel. Sobre ella se ejecuta ROS 2 en su distribución *Humble*, una versión de soporte prolongado que es la recomendada para el TurtleBot 4 (Macenski et al., 2022).

La navegación se construye sobre tres paquetes consolidados del ecosistema. La pila Nav2 se ocupa de la planificación y el control del movimiento (Macenski et al., 2020); `slam_toolbox` construye el mapa del entorno mediante SLAM láser (Macenski & Jambrecic, 2021); y `explore_lite` dirige la exploración autónoma inicial que recorre el espacio desconocido hasta completar el mapa. Una vez construido y guardado el mapa, los arranques posteriores operan en modo de localización sobre él, con Nav2 apoyándose en AMCL en lugar de en SLAM. Reutilizar estas soluciones, en lugar de reimplementar la navegación, permite concentrar el trabajo en la orquestación y la integración del sistema.

4.2.2 Percepción visual

La detección de personas se realiza con YOLO en su versión *nano* (YOLOv8n), a través de la biblioteca de Ultralytics (Jocher et al., 2023). Se ha optado por la variante *nano* por ser la más ligera de la familia, lo que permite ejecutar la inferencia en la CPU del propio robot a una velocidad suficiente para la patrulla, sin necesidad de una GPU dedicada. La biblioteca aporta, además, modelos preentrenados y una interfaz sencilla que reduce el coste de integración. El tratamiento de las imágenes, esto es, la captura de los fotogramas de la cámara y la codificación del clip de vídeo de cada incidencia, se apoya en OpenCV, biblioteca estándar de facto en visión por computador.

4.2.3 Mensajería orientada a eventos

El bus de eventos que comunica el robot con el servidor es Apache Kafka (Kreps et al., 2011). Se ha preferido a una solución de mensajería más ligera porque conserva los mensajes en un registro durable, lo que desacopla por completo al productor del consumidor y permite que el backend procese la telemetría y las incidencias a su propio ritmo, e incluso reprocese eventos pasados. La comunicación se organiza en tres *topics* (`robot_telemetry`, `robot_incidents` y `robot_commands`). Tanto el robot como el backend acceden a Kafka mediante `confluent-kafka`, un cliente de alto rendimiento basado en la biblioteca `librdkafka`, escrita en C.

4.2.4 Backend y servicios web

El backend se ha desarrollado con FastAPI, un framework web asíncrono para Python. Su elección responde a tres motivos: el alto rendimiento que aporta su modelo de servidor asíncrono (*ASGI*), la validación automática de los datos de entrada y salida mediante Pydantic, y la generación automática de la documentación de la API según el estándar OpenAPI. La aplicación se sirve con Uvicorn, un servidor *ASGI*, y la configuración (credenciales y direcciones de los servicios) se gestiona con `pydantic-settings` a partir de variables de entorno, lo que facilita el despliegue en distintos equipos.

El acceso a la base de datos relacional se realiza mediante el Object-Relational Mapping (ORM) SQLAlchemy, que abstrae las consultas Structured Query Language (SQL) y mapea las tablas a clases de Python, mientras que Alembic versiona y aplica los cambios del esquema. El almacenamiento de los vídeos se gestiona con `boto3`, el Software Development Kit (SDK) de Simple Storage Service (S3), que permite operar sobre MinIO con la misma interfaz que el servicio de Amazon. Por último, la seguridad se apoya en `pyjwt` para los tokens JWT y en `passlib` con `bcrypt` para el cifrado de las contraseñas de los usuarios.

4.2.5 Persistencia, almacenamiento y visualización

La distinta naturaleza de los datos del sistema ha llevado a emplear un almacén específico para cada tipo. Los datos estructurados y con relaciones (usuarios, robots e incidencias) residen en PostgreSQL, una base de datos relacional madura que garantiza la integridad mediante transacciones. La telemetría del robot (batería, posición y estado), de carácter temporal y alta frecuencia, se almacena en InfluxDB, una base de datos de series temporales (Time-Series Database (TSDB)) optimizada para este tipo de datos y para las consultas por intervalos de tiempo. El vídeo de las incidencias, al ser un objeto binario de gran tamaño, se guarda en MinIO, un almacén de objetos compatible con la API de S3; esta separación evita sobrecargar la base de datos relacional con ficheros pesados. Finalmente, Grafana proporciona la visualización de la telemetría mediante paneles que consultan InfluxDB, con un panel por robot que la aplicación de usuario incrusta directamente.

4.2.6 Aplicación de usuario

La aplicación con la que interactúa el usuario se ha construido con Flutter, el framework de Google basado en el lenguaje Dart. Su principal ventaja es permitir desarrollar, a partir de una única base de código, una aplicación para móvil, web y escritorio, algo idóneo para un trabajo con tiempo limitado. Sobre Flutter se apoyan varios paquetes especializados:

la gestión del estado se resuelve con `flutter_riverpod`; la navegación entre pantallas, con `go_router`; las llamadas a la API REST del backend, con el cliente HTTP `dio`; la persistencia local de la sesión, con `shared_preferences`; los componentes de la interfaz, con `shadcn_ui`; y la apertura del panel de Grafana en el navegador, con `url_launcher`.

4.2.7 Infraestructura y despliegue

Toda la infraestructura de datos (PostgreSQL, Kafka, InfluxDB, MinIO y Grafana) se ejecuta en contenedores gestionados con Docker y orquestados mediante Docker Compose. Esta decisión persigue la reproducibilidad del entorno: un único fichero declarativo describe todos los servicios, sus versiones y su configuración, de modo que el sistema completo puede levantarse en cualquier equipo con un solo comando, sin instalaciones manuales y con un comportamiento idéntico. El aislamiento entre contenedores evita, además, los conflictos de dependencias entre los distintos servicios.

4.2.8 Entorno de desarrollo y herramientas de apoyo

El grueso del sistema se ha desarrollado en **Python**, lenguaje común al backend y a los nodos del robot, lo que ha permitido compartir conocimientos y bibliotecas entre ambas partes. Es, además, el lenguaje de referencia para la programación de nodos en ROS 2 y para el ecosistema de visión por computador y aprendizaje automático empleado en la percepción. Para aislar las dependencias de cada componente se han utilizado entornos virtuales (*virtual environments*, `venv`), de modo que el backend y la capa robótica mantienen conjuntos de paquetes independientes y sin interferencias entre versiones; las dependencias de cada entorno se declaran de forma explícita (por ejemplo, en el fichero `requirements.txt` del backend), lo que refuerza la reproducibilidad ya mencionada.

Durante la elaboración del trabajo se ha empleado, asimismo, el asistente de inteligencia artificial Claude como herramienta de apoyo en tareas de revisión, búsqueda de información (*research*) y documentación. Su uso se ha limitado en todo momento a un papel auxiliar: las decisiones de diseño del sistema han sido tomadas y validadas por el autor, sin que la herramienta las sustituya.

4.2.9 Bibliotecas y dependencias principales

A modo de síntesis, la Tabla 4.2 recoge las bibliotecas más relevantes empleadas en el desarrollo, indicando su función y el componente del sistema en el que se utilizan. La mayoría se han presentado ya en las subsecciones anteriores; la tabla las reúne como referencia.

Tabla 4.2: Bibliotecas y dependencias principales, su función y el componente en el que se emplean. Fuente: elaboración propia.

Biblioteca	Función
<i>Backend (Python)</i>	
FastAPI	Framework web asíncrono y exposición de la API REST
Uvicorn	Servidor <i>ASGI</i> que ejecuta la aplicación
SQLAlchemy	ORM de acceso a PostgreSQL
Alembic	Migraciones del esquema de la base de datos
influxdb-client	Escritura de la telemetría en InfluxDB
confluent-kafka	Consumo de los eventos del bus Kafka
boto3	Acceso a MinIO (S3) para el vídeo de incidencias
Pydantic	Validación de datos y configuración por entorno
PyJWT	Emisión y verificación de tokens JWT
passlib / bcrypt	Cifrado de las contraseñas de usuario
httpx	Cliente HTTP asíncrono
<i>Capa robótica (robot_ws, Python sobre ROS 2)</i>	
Ultralytics (YOLO)	Detección de personas durante la patrulla
OpenCV	Captura de fotogramas y codificación del clip de vídeo
NumPy	Cálculo numérico de soporte a la percepción
confluent-kafka	Publicación de telemetría e incidencias en Kafka
boto3	Subida de los clips de vídeo a MinIO
httpx	Solicitud del token al backend (TOFU y JWT)
<i>Aplicación de usuario (Flutter, Dart)</i>	
Dio	Cliente HTTP para consumir la API REST
Riverpod	Gestión del estado de la aplicación
go_router	Navegación entre pantallas
shared_preferences	Persistencia local de la sesión
shadcn_ui	Componentes de la interfaz de usuario
url_launcher	Apertura del panel de Grafana en el navegador
intl	Formato de fechas y localización

4.3 Criterios y métricas de evaluación

Dado que el sistema integra subsistemas de naturaleza distinta, su evaluación contempla varias dimensiones complementarias:

- **Navegación y patrulla.** Capacidad del robot para construir el mapa, generar una ruta de cobertura válida y recorrerla de forma cíclica sin colisiones, así como la tasa de éxito en alcanzar las poses objetivo.
- **Detección de incidencias.** Acierto del detector en la identificación de personas, considerando la confianza de las detecciones y la presencia de falsos positivos y falsos negativos.
- **Rendimiento del sistema de extremo a extremo.** Latencia desde que se produce un evento en el robot hasta que la incidencia queda registrada y es visible para el usuario, así como la regularidad del envío de telemetría.
- **Validación funcional de la integración.** Comprobación de que los flujos completos (arranque, mapeado, patrulla, detección, notificación y control desde la aplicación) se ejecutan de principio a fin.

Conviene señalar que, en el estado actual del trabajo, la evaluación realizada es fundamentalmente de carácter **funcional**. La obtención de métricas cuantitativas sistemáticas, especialmente sobre hardware real, queda pendiente y se discute en el Capítulo 6 y entre las líneas de trabajo futuro del Capítulo 7.

5 Desarrollo

En este capítulo se detalla la implementación del sistema, organizada en torno a los flujos de datos que lo recorren de extremo a extremo. Se parte de una visión global por capas (sección 5.1) y, a continuación, se desciende al detalle de la infraestructura de despliegue (sección 5.2), del backend (sección 5.3), de la capa robótica (sección 5.4) y de la aplicación de usuario (sección 5.5). La descripción se ancla en el código fuente del proyecto, del que se incluyen extractos representativos.

5.1 Vista general y flujos de datos

El sistema se estructura en seis capas, desde la aplicación de usuario hasta la plataforma física del robot. La figura 5.1 las presenta apiladas, junto con los componentes que las integran. El flujo de información es bidireccional: las órdenes del usuario descienden desde la aplicación hasta la plataforma física, mientras que la telemetría y las incidencias ascienden en sentido contrario.

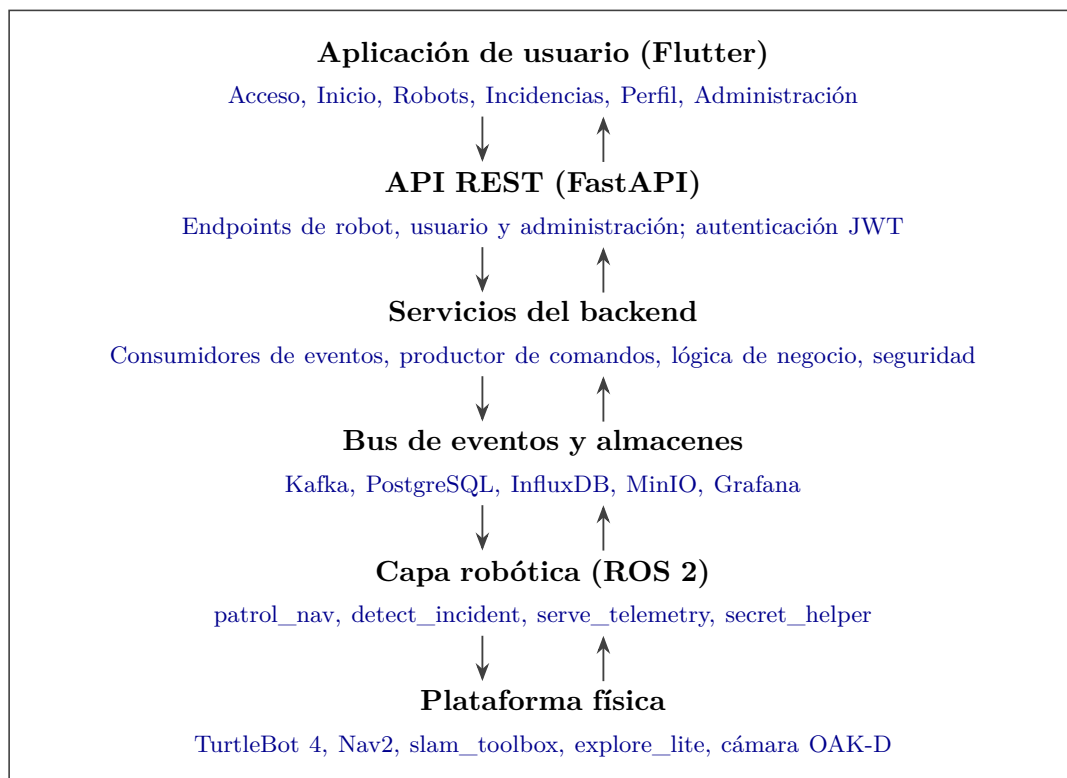


Figura 5.1: Vista general del sistema organizada en capas, desde la aplicación de usuario hasta la plataforma física del robot. Fuente: elaboración propia.

Los apartados siguientes describen los tres flujos principales que atraviesan estas capas.

5.1.1 Cliente → servidor → robot: control de la patrulla

Las órdenes del usuario (iniciar, detener y reinstalar la patrulla) viajan desde la aplicación hasta el robot a través del backend y del bus de eventos. La aplicación invoca el endpoint correspondiente (PUT `/user/{id}/start-patrol`, `stop-patrol` o `reinstall`). El backend valida que el robot pertenece al usuario autenticado y que su estado operativo permite la transición; a continuación publica un comando en el topic `robot_commands` de Kafka. En el robot, el nodo orquestador `patrol_nav` sondea dicho topic cada 0,5s (`poll_command_topic`), descarta los mensajes cuyo `robot_id` no coincide con el suyo y, según el comando, ejecuta `handle_start_patrol`, `handle_stop_patrol` o `handle_reinstall`, provocando la transición de su máquina de estados.

Cada comando responde a una intención del usuario distinta (el detalle de las transiciones se aborda en la sección 5.4.1). **Iniciar** pone en marcha la patrulla (el robot se desacopla de su base antes de moverse): si el robot no dispone todavía de un mapa, dispara la exploración y el mapeo del entorno antes de empezar a patrullar; si ya existe un mapa guardado, lo reutiliza y pasa directamente a planificar la ruta y a recorrerla. **Detener** cancela la navegación en curso, devuelve el robot al *dock*, donde se acopla, y deja la patrulla en pausa. **Reinstalar** borra el mapa guardado y devuelve el robot a su estado inicial, de modo que el siguiente inicio vuelve a explorar el entorno desde cero.

Conviene precisar cómo se gobierna el estado en la base de datos. La orden de inicio no escribe el estado operativo en la base de datos: deja esa responsabilidad al *heartbeat* del robot, de modo que la primera vuelta no se salte la fase de mapeo. En cambio, las órdenes de detención y reinstalación sí fijan de forma optimista los valores `stopped` e `idle`, respectivamente. En todos los casos, el *heartbeat* periódico del robot (sección 5.1.2) actúa como fuente de verdad que reconcilia el estado almacenado con el estado real.

Esta delegación introduce una ventana de consistencia eventual acotada. La aplicación no realiza una actualización optimista de la interfaz: tras enviar la orden vuelve a consultar el estado, que puede seguir mostrando el valor previo hasta que el siguiente *heartbeat* (a lo sumo 10s después) actualice la base de datos y el sondeo periódico de la aplicación (cada 5s) lo refleje. Se trata, por tanto, de un retardo breve y acotado, no de una inconsistencia permanente, ya que el *heartbeat* garantiza la convergencia al estado real.

5.1.2 Robot → servidor → cliente: telemetría, incidencias y estado

En sentido ascendente coexisten tres caminos.

Telemetría. El nodo `serve_telemetry` publica el estado del robot una vez por segundo en el topic `robot_telemetry`, incorporando el JWT del robot en la cabecera `Authorization` del mensaje. En el backend, un consumidor en hilo independiente verifica el token, comprueba que el `robot_id` del mensaje coincide con el del token y escribe un punto de la medida `robot_telemetry` en InfluxDB. Grafana consume esa serie temporal y la aplicación la muestra empotrada en modo *kiosk*, obteniendo la URL filtrada por robot mediante GET `/user/{id}/dashboard-url`.

Incidencias. El nodo `detect_incident` detecta personas mediante visión por computador, graba un vídeo del suceso, lo sube a MinIO y publica en el topic `robot_incidents` (de nuevo con el JWT en cabecera) los datos del objeto almacenado. El consumidor de incidencias del backend valida el token, crea un registro `Incidente` en PostgreSQL y lo deja disponible para la aplicación, que lista las incidencias (`GET /user/incidents`) y reproduce el vídeo a través de una URL prefirmada (`GET /user/incidents/{id}/show-video`).

Estado operativo. El orquestador `patrol_nav` envía un *heartbeat* cada 10s mediante `PUT /robot/heartbeat`, traduciendo su estado interno de la Finite State Machine (FSM) a uno de los valores admitidos. El backend valida el valor recibido y actualiza la conexión y el estado operativo del robot en PostgreSQL, datos que la aplicación refleja en la pantalla de inicio.

5.1.3 Autenticación y arranque: TOFU y JWT

Antes de poder emitir telemetría o incidencias, el robot debe autenticarse. El nodo `secret_helper` aplica el patrón TOFU para acordar un secreto con el backend en el primer arranque y, a partir de ese secreto, obtener un JWT que renueva de forma periódica. El resto de nodos consume ese token sin gestionar credenciales por su cuenta. El mecanismo se describe en detalle en la sección 5.4.4.

La tabla 5.1 resume los flujos descritos.

Tabla 5.1: Resumen de los flujos de datos del sistema. Fuente: elaboración propia.

Flujo	Sentido	Recorrido principal	Persistencia
Control de patrulla	App → Robot	REST → Kafka <code>robot_commands</code> → FSM	PostgreSQL
Telemetría	Robot → App	Kafka <code>robot_telemetry</code> → InfluxDB → Grafana	InfluxDB
Incidencias	Robot → App	MinIO + Kafka <code>robot_incidents</code> → <i>backend</i>	MinIO, PostgreSQL
Estado operativo	Robot → App	<code>PUT /robot/heartbeat</code>	PostgreSQL
Autenticación	Robot ↔ Backend	TOFU: <code>first-boot</code> / <code>auth</code> → JWT	PostgreSQL

5.2 Arquitectura e infraestructura

La infraestructura se despliega mediante contenedores. El fichero `docker-compose.yml` del backend define ocho servicios: la base de datos relacional (`postgres`), el bus de eventos (`zookeeper` y `kafka`) con su trabajo de inicialización (`init-kafka`), la base de series temporales (`influxdb`), el almacén de objetos (`minio`) con su inicializador (`init-minio`) y la visualización (`grafana`). Por su parte, el `docker-compose.yml` de `robot_ws` define un único servicio de desarrollo ROS 2 (`ros2_dev`), conectado a la red externa `backend_default`.

Conviene matizar que este `docker-compose` no constituye el despliegue de producción del robot, sino un entorno de desarrollo y simulación: proporciona, con acceso a WSLg y GPU, las herramientas gráficas (RViz y Gazebo principalmente) necesarias para ejecutar pruebas y lanzar simulaciones sin depender del hardware. En una instalación real, los nodos descritos en la sección 5.4 se ejecutan directamente sobre la plataforma física del TurtleBot 4.

Es importante señalar que ni el backend (*FastAPI*) ni los nodos del robot se ejecutan dentro de los contenedores de infraestructura: corren como procesos externos y se conectan a los servicios anteriores a través de los puertos publicados en el anfitrión. La tabla 5.2 recoge esos puertos.

Tabla 5.2: Servicios de infraestructura y puertos publicados. Fuente: elaboración propia.

Servicio	Imagen	Puerto	Función
postgres	postgres:17	5432	Datos relacionales
zookeeper	cp-zookeeper:7.5.0	2181 (int.)	Coordinación Kafka
kafka	cp-kafka:7.5.0	9094 / 29092	Bus de eventos
influxdb	influxdb:2.7	8086	Series temporales
minio	minio (2024)	9000 / 9001	Objetos / consola
grafana	grafana:latest	3000	Visualización
backend	FastAPI (proceso)	8000	API REST
nodos robot	ROS 2 (procesos)	n/d	Cliente del bus

La topología de red de Kafka merece una nota. El *broker* expone dos *listeners*: uno interno, en el puerto 29092, para los clientes que viven dentro de la red de contenedores, y otro externo, en el puerto 9094, para el backend y los nodos del robot cuando corren en el anfitrión. El extracto 5.1 muestra esa configuración, en la que `<IP-DEL-HOST>` representa la dirección real del anfitrión. Los trabajos de inicialización crean, respectivamente, los tres tópicos (`robot_incidents`, `robot_telemetry`, `robot_commands`) y el *bucket* `tfg-incidentes` de MinIO. Por coherencia, los nodos del robot disponen de dos ficheros de entorno: `.env` para ejecución en el anfitrión y `.env.container` con las direcciones internas (`kafka:29092`, `minio:9000`, `host.docker.internal:8000`).

Listeners del broker Kafka (docker-compose.yml)

```

1 kafka:
2   image: confluentinc/cp-kafka:7.5.0
3   environment:
4     KAFKA_ADVERTISED_LISTENERS:
      ↪ PLAINTEXT://kafka:29092,PLAINTEXT_HOST://<IP-DEL-HOST>:9094
5     KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
      ↪ PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
6     KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
7   ports:
8     - "9094:9094"
```

5.3 Backend

El backend, construido con *FastAPI*, expone la API REST, gestiona la persistencia y aloja los consumidores de eventos. Se organiza en módulos: `models.py` (modelo de datos), `server.py` (endpoints y consumidores), `security.py` (criptografía y tokens) y `dependencies.py` (inyección de identidad).

5.3.1 Modelo de datos

El modelo, definido con el ORM *SQLAlchemy* en `models.py`, consta de tres entidades. `Usuario` tiene clave primaria entera autoincremental, `email` único e indexado, un rol (`user` o `admin`, por defecto `user`) y la contraseña almacenada como `password_hash`. `Robot` usa una clave primaria de tipo cadena (`String(50)`, apta para números de serie), un `secret_hash` inicialmente nulo (se genera en el primer arranque TOFU), una clave foránea `usuario_id` hacia su propietario y los campos `estado_conexion` (`online/offline`) y `estado_operativo`; conserva además marcas temporales de creación y de última actividad, esta última actualizada en cada escritura mediante `onupdate`. `Incidente` emplea como clave primaria un identificador UUID generado aleatoriamente (`uuid4`), lo que dificulta la enumeración de recursos por fuerza bruta, junto con `bucket_name`, `video_filename` y el indicador `revisado`.

La elección de los tipos de clave primaria no es accidental: que el usuario tenga un identificador numérico y el robot una cadena no numérica es lo que permite distinguir a ambos a partir del sujeto (`sub`) del token, sin necesidad de un campo de tipo adicional (sección 5.3.4). El borrado se propaga en cascada (`onDelete=CASCADE`): al eliminar un usuario se eliminan sus robots y, con ellos, sus incidencias. Para acelerar las consultas más frecuentes, la tabla de incidencias declara dos índices: uno compuesto por robot y fecha (`idx_incidentes_robot_fecha`), que sirve al listado cronológico por robot, y otro parcial sobre las no revisadas (`idx_incidentes_no_revisados`, con condición `revisado = false`), que se mantiene compacto al cubrir únicamente las incidencias pendientes de atención.

La persistencia se organiza en torno a `SessionLocal`, una *fábrica de sesiones* de *SQLAlchemy* creada con `sessionmaker`: cada llamada a `SessionLocal()` produce una sesión nueva, es decir, una conversación aislada con la base de datos. Se configura con `autocommit` y `autoflush` desactivados, lo que otorga control explícito sobre la transacción: ningún cambio se envía ni se confirma de forma implícita, sino solo cuando el código lo ordena con `commit`, de modo que una petición que falle a mitad no deja datos a medio escribir. Para que cada petición HyperText Transfer Protocol (HTTP) disponga de su propia sesión y esta se cierre siempre al terminar, el acceso se encapsula en la dependencia `get_db`, que *FastAPI* inyecta en cada endpoint que la declara. Los consumidores de eventos, que viven fuera del ciclo de petición, gestionan su sesión por su cuenta con `SessionLocal()`.

5.3.2 API REST

La tabla 5.3 recoge el inventario completo de endpoints, agrupados según la dependencia de autenticación que exigen, y no según la etiqueta de documentación, que en algunos casos resulta engañosa.

La autenticación se resuelve por inyección de dependencias de *FastAPI*. Un esquema `HTTPBearer` extrae el token de la cabecera `Authorization` y tres dependencias lo tradu-

Tabla 5.3: Endpoints de la API REST agrupados por ámbito de autenticación. Fuente: elaboración propia. Se omiten los endpoints base GET / y GET /health.

Método	Ruta	Autenticación
<i>Ámbito robot</i>		
PUT	/robot/heartbeat	JWT robot
POST	/robot/{id}/first-boot	Ninguna (TOFU)
POST	/robot/{id}/auth	secret_key
<i>Ámbito usuario</i>		
POST	/user/register	Ninguna
POST	/user/login	Ninguna
GET	/user/profile	JWT usuario
POST	/user/new-robot	JWT usuario
GET	/user/robots	JWT usuario
DELETE	/user/{id}/delete	JWT usuario
GET	/user/{id}/dashboard-url	JWT usuario
PUT	/user/{id}/start-patrol	JWT usuario
PUT	/user/{id}/stop-patrol	JWT usuario
PUT	/user/{id}/reinstall	JWT usuario
PUT	/user/{id}/alias	JWT usuario
GET	/user/incidents	JWT usuario
GET	/user/{id}/incidents	JWT usuario
GET	/user/incidents/{id}	JWT usuario
PUT	/user/incidents/{id}/review	JWT usuario
GET	/user/incidents/{id}/show-video	JWT usuario
PUT	/user/update-password	JWT usuario
PUT	/user/update-info	JWT usuario
<i>Ámbito administración</i>		
GET	/admin/robots	JWT admin
GET	/admin/robots/{id}/incidents	JWT admin
GET	/admin/incidents/{id}	JWT admin
GET	/admin/incidents/{id}/show-video	JWT admin
PUT	/admin/incidents/{id}/review	JWT admin
GET	/admin/users	JWT admin
DELETE	/admin/users/{id}/delete	JWT admin
DELETE	/admin/robots/{id}/delete	JWT admin
PUT	/admin/users/{id}/grant	JWT admin
PUT	/admin/users/{id}/revoke	JWT admin

cen a identidad: `get_current_user` y `get_current_robot` decodifican el token y cargan la entidad correspondiente, mientras que `get_current_admin` encadena la primera y además exige rol de administrador. Cada endpoint declara la dependencia que necesita, de modo que el control de acceso queda expresado en la propia firma de la función y *FastAPI* rechaza la petición antes de ejecutar el cuerpo si la identidad no es la adecuada.

Los endpoints de control ilustran el patrón completo. `PUT /user/{id}/start-patrol` comprueba primero que el robot pertenece al usuario autenticado (en caso contrario responde 403 *Forbidden*) y que su `estado_operativo` admite la transición: solo se puede iniciar desde `idle` o `stopped`, y cualquier otro estado devuelve 409 *Conflict* con un mensaje explicativo. Superadas las validaciones, el endpoint publica en el topic `robot_commands` un mensaje JavaScript Object Notation (JSON) con la estructura `{robot_id, command, timestamp}`, usando el `robot_id` como clave de partición y forzando la entrega con `flush` (tiempo límite de cinco segundos); si el *broker* no está disponible, responde 503 *Service Unavailable*. `stop-patrol` aplica la misma plantilla pero solo admite la transición desde `patrolling`, y `reinstall` no impone restricción de estado.

Aquí se materializa la asimetría descrita en la sección 5.1.1. En `start-patrol`, la línea que escribiría el estado en la base de datos se ha dejado deliberadamente comentada en el código: tras publicar el comando en Kafka, el endpoint devuelve el robot sin tocar su `estado_operativo`, de manera que sea el *heartbeat* quien refleje la progresión real `exploring` → ... → `patrolling` y la primera vuelta no se salte la fase de mapeo. En cambio, `stop-patrol` y `reinstall` sí fijan de forma optimista `stopped` e `idle`, respectivamente, porque son estados terminales que no dependen de un proceso ulterior del robot.

Dos endpoints de lectura completan el flujo del usuario. `GET /user/{id}/dashboard-url` construye, tras validar la propiedad del robot, la URL del panel de Grafana en modo *kiosk* filtrada por el robot (`var-robot_id`), con ventana temporal de las últimas seis horas y refresco cada diez segundos. `GET /user/incidents/{id}/show-video` genera con *boto3* una URL prefirmada del objeto de MinIO con validez de una hora (`ExpiresIn=3600`), de manera que la aplicación reproduce el vídeo sin exponer las credenciales del almacén ni hacer público el *bucket*.

5.3.3 Consumidores y productor de eventos

En `server.py`, la función `create_server_consumers` instancia dos consumidores de Kafka y un productor para los comandos. Cada consumidor pertenece a un grupo propio (`incident-consumer` y `telemetry-consumer`) y difiere en su política de *offset* inicial, una decisión deliberada: el de incidencias arranca desde el mensaje más antiguo (`auto.offset.reset = earliest`) para no perder ningún suceso aunque el servidor haya estado caído, mientras que el de telemetría arranca desde el más reciente (`latest`), porque una medida atrasada carece de valor y solo interesa el estado actual del robot.

Al iniciarse el servidor, cada bucle de consumo arranca en un hilo *daemon* independiente (extracto 5.2), de modo que la espera de mensajes no bloquea al hilo principal que atiende las peticiones HTTP y los consumidores se detienen solos al apagar el servidor. El productor de comandos se guarda en `app.state` para que lo compartan los endpoints de control.

Cada mensaje recibido pasa por la misma cadena de validación antes de procesarse (sección 5.3.4): se verifica el JWT de la cabecera y se comprueba que el `robot_id` del cuerpo coincide con el sujeto del token, descartando el mensaje en cualquier otro caso.

Consumidores en hilos daemon y sondeo del broker (server.py)

```

1 # Distinta politica de offset inicial por consumidor.
2 incident_consumer = Consumer({
3     'bootstrap.servers': settings.kafka_broker_url,
4     'group.id': 'incident-consumer',
5     'auto.offset.reset': 'earliest'}) # No perder sucesos pasados.
6 telemetry_consumer = Consumer({
7     'bootstrap.servers': settings.kafka_broker_url,
8     'group.id': 'telemetry-consumer',
9     'auto.offset.reset': 'latest'}) # Solo interesa el estado actual.
10
11 # Cada bucle corre en un hilo daemon: no bloquea el apagado ni las peticiones
12   ↳ HTTP.
13 threading.Thread(target=detect_incident,
14                  args=(incident_consumer,), daemon=True).start()
15 threading.Thread(target=serve_telemetry,
16                  args=(telemetry_consumer,), daemon=True).start()
17
18 # Dentro de cada bucle: espera <= 1 s por mensaje, sin espera activa.
19 while True:
20     msg = consumer.poll(1.0)
21     if msg is None:
22         continue # No habia mensajes en este
23         ↳ intervalo.
24 # ... validacion del JWT y del robot_id antes de procesar ...

```

El tratamiento posterior difiere según el flujo. El consumidor de telemetría compone un punto de la medida `robot_telemetry`, etiquetado por `robot_id` y con los campos de estado del robot (batería y voltaje, posición, velocidades lineal y angular, e indicadores de atraque, secuestro, deslizamiento y ruedas), y lo escribe en InfluxDB con la marca temporal que envía el propio robot, a precisión de nanosegundos. El consumidor de incidencias abre una sesión de base de datos por mensaje, inserta una fila `Incidente` con los datos del objeto almacenado y confirma la transacción, revirtiéndola si se produce un error. Al apagarse, `shutdown_event` vacía el productor para no perder los comandos pendientes de entrega.

5.3.4 Seguridad

La seguridad se apoya en varios mecanismos complementarios concentrados en `security.py` y `dependencies.py`. El esquema parte de una premisa: el robot opera sin supervisión y no puede depender de que una persona introduzca credenciales, de modo que acuerda un secreto con el backend en su primer contacto (TOFU) y, a partir de él, porta su identidad en cada petición y evento mediante un JWT. Las contraseñas de usuario y los secretos de robot nunca se almacenan en claro: se cifran de forma irreversible con `bcrypt` a través de `passlib` (un `CryptContext` con el esquema `bcrypt`), que incorpora `salt` y un factor de coste configurable. El secreto raíz del robot, base del TOFU, se genera con el módulo `secrets` de Python como una cadena aleatoria de treinta y dos caracteres alfanuméricos con un prefijo identificativo (`rbt_sec_`), y de él solo se persiste el `hash`.

La identidad se transporta en JWT firmados con el algoritmo simétrico HS256. El *payload* contiene únicamente el sujeto (*sub*) y la fecha de expiración (*exp*), fijada por defecto a 1440 minutos (veinticuatro horas). La función `verificar_token` valida la firma y la vigencia y traduce los fallos a respuestas 401, distinguiendo el token caducado (`ExpiredSignatureError`) del inválido o corrupto (`InvalidTokenError`). La distinción entre usuarios y robots se resuelve sin almacenar el tipo: se observa si el sujeto del token es numérico, ya que los identificadores de usuario lo son y los de robot (cadenas) no. Sobre esta base, `get_current_user` rechaza con 403 un token cuyo sujeto no sea numérico y `get_current_robot` hace lo contrario, evitando que una credencial de robot opere como usuario o viceversa. El control de acceso por roles (Role-Based Access Control (RBAC)) se aplica en `get_current_admin`, que encadena la verificación de usuario y exige rol de administrador; además, los endpoints de administración protegen contra operaciones peligrosas sobre la propia cuenta (un administrador no puede borrarse ni revocarse a sí mismo).

Para los mensajes que llegan por Kafka, que no atraviesan las dependencias de *FastAPI*, el backend replica la verificación en el propio bucle de consumo: valida el JWT incluido en la cabecera del mensaje y comprueba que el `robot_id` del cuerpo coincide con el sujeto del token, evitando la suplantación de un robot por otro. El acceso a los vídeos se realiza mediante URL prefirmadas de MinIO generadas con *boto3* y caducidad de una hora, de modo que el *bucket* permanece privado. Conviene señalar de forma explícita una limitación conocida: la política de Cross-Origin Resource Sharing (CORS) permanece abierta en el estado actual del desarrollo y debe restringirse antes de un despliegue en producción. El extracto 5.3 muestra la generación y verificación de tokens.

Generación y verificación de JWT (security.py)

```

1 def generate_jwt_token(subject: str) -> str:
2     tiempo_expiracion = datetime.now(timezone.utc) + timedelta(
3         minutes=settings.access_token_expire_minutes)
4     payload = {"sub": str(subject), "exp": tiempo_expiracion}
5     return jwt.encode(payload, settings.secret_key,
6                       algorithm=settings.algorithm)
7
8 def verificar_token(token: str) -> str:
9     payload = jwt.decode(token, settings.secret_key,
10                          algorithms=[settings.algorithm])
11     subject = payload.get("sub")
12     if not subject:
13         raise HTTPException(status_code=401, detail="Token sin ID (subject)")
14     return subject

```

5.4 Capa robótica

La capa robótica es un espacio de trabajo de ROS 2 Humble que combina paquetes propios y de terceros. Los paquetes propios son cuatro nodos, `patrol_nav`, `detect_incident`, `serve_telemetry` y `secret_helper`, más el paquete de interfaces `secret_helper_interfaces`, que define el servicio `GetToken`. Los de terceros son `explore_lite` (Alvarez, 2022) para la

exploración, `slam_toolbox` (Macenski & Jambrecic, 2021) para el mapeo, Nav2 (Macenski et al., 2020) para la navegación, `irobot_create_msgs` para las acciones de acople y desacople de la base, y *Ultralytics* YOLO (Jocher et al., 2023; Redmon et al., 2016) para la visión por computador. La mensajería con el backend usa Kafka (Kreps et al., 2011). La tabla 5.4 resume los nodos propios.

Tabla 5.4: Nodos ROS 2 propios del sistema. Fuente: elaboración propia.

Nodo	Responsabilidad	Interfaces
<code>patrol_nav</code>	Orquestación de la patrulla (FSM)	<code>robot_commands</code> , <code>/odom</code> , <code>/dock_status</code> , Nav2, <code>dock/undock</code>
<code>detect_incident</code>	Detección de personas y grabación	<code>/oakd/rgb/...</code> , <code>robot_incidents</code>
<code>serve_telemetry</code>	Recolección y envío de telemetría	sensores, <code>robot_telemetry</code>
<code>secret_helper</code>	TOFU y refresco del JWT	servicio <code>get_robot_token</code>

5.4.1 Orquestador: `patrol_nav`

El nodo `patrol_nav` implementa una máquina de estados finitos (tabla 5.5, con las transiciones de la figura 5.2) que arranca en `IDLE` a la espera del primer comando de inicio. Antes de enviar ningún objetivo de navegación, el nodo debe asegurarse de que Nav2 ha terminado de arrancar, para lo que emplea la utilidad `waitUntilNav2Active`. La forma de esperar depende de si el robot ya dispone de un mapa guardado, lo que da lugar a dos configuraciones. En la primera vuelta, sin mapa, el robot navega con SLAM, que construye el mapa y estima la posición sobre la marcha; en esa configuración no existe el nodo de localización `amcl`, y esperarlo bloquearía el arranque de forma indefinida, así que el nodo indica a `waitUntilNav2Active` que se fije únicamente en el motor de navegación `bt_navigator`, que es el componente que realmente ejecuta los objetivos. Cuando ya existe un mapa guardado, el robot opera en modo de localización: Nav2 arranca con un servidor de mapa y `amcl`, el nodo fija la pose inicial en el `dock` con `setInitialPose` y espera la disponibilidad completa de la pila (`amcl` y `bt_navigator`) con el comportamiento por defecto de `waitUntilNav2Active`.

El lanzador `patrol_bringup.launch.py` elige el modo de forma automática a partir de la existencia del fichero del mapa, sin que haya que fijarlo a mano: si el mapa existe, arranca Nav2 en modo de localización (servidor de mapa y `amcl`); si no, en modo SLAM (exploración y mapeo), levantando además en este caso el servicio de guardado del mapa. El argumento `sim` del lanzador selecciona el entorno de ejecución: con valor verdadero arranca el simulador Ignition del TurtleBot 4 y, con valor falso, la pila de navegación sobre el robot real. Así, tras una reinstalación que borra el mapa, el siguiente lanzamiento vuelve a empezar en SLAM sin intervención manual.

Superada esta comprobación, el nodo se organiza en torno a cuatro temporizadores: uno evalúa la máquina de estados a 1 Hz (`state_machine`), otro sondea el topic de comandos cada 0,5s, un tercero emite el `heartbeat` cada 10s (traduciendo el estado interno con el diccionario

FSM_TO_OPERATIVO) y el último refresca el token cada 10s. El sondeo de comandos emplea un consumidor de Kafka propio, con grupo dependiente del `robot_id` y `offset` inicial `latest` (atiende solo órdenes nuevas); descarta los mensajes dirigidos a otros robots y despacha `start_patrol`, `stop_patrol` o `reinstall` al manejador correspondiente. Todos los nodos del espacio de trabajo se ejecutan sobre un *executor* de un solo hilo, lo que obliga a que las operaciones potencialmente bloqueantes (como la obtención del token) se realicen de forma asíncrona para no detener el resto de temporizadores.

Tabla 5.5: Estados de la máquina de estados de `patrol_nav`. Fuente: elaboración propia.

Estado FSM	Heartbeat	Significado
IDLE	<code>idle</code>	Registrado, sin mapa o recién arrancado
EXPLORING	<code>exploring</code>	Exploración autónoma del entorno
SAVING_MAP	<code>saving_map</code>	Guardado del mapa generado
PLANNING	<code>planning</code>	Cálculo de la ruta de patrulla
PATROLLING	<code>patrolling</code>	Recorrido cíclico de los waypoints
PATROL_STOPPED	<code>stopped</code>	Patrulla detenida, robot acoplado en su base
UNDOCKING	<code>planning</code>	Desacople de la base antes de conducir
DOCKING	<code>stopped</code>	Acople final en la base al detener la patrulla

Conviene notar que estos dos estados de transición no amplían el vocabulario de estado operativo que conoce el servidor: `UNDOCKING` se comunica como `planning` y `DOCKING` como `stopped`, de modo que el acople y el desacople quedan integrados en los estados de planificación y parada ya existentes.

El ciclo completo abarca varias fases. Antes de cualquier estado que implique conducir, el nodo comprueba si el robot está acoplado (indicador `is_docked`, que mantiene la suscripción a `/dock_status`) y, en ese caso, antepone un **desacople** (estado `UNDOCKING`): como Nav2 no puede mover el robot mientras está en la base, el método `_begin` recuerda el destino, lanza la acción `undock` de la base *iRobot Create* y, al completarse, continúa hacia él. En **exploración**, el nodo lanza `explore_lite` como subproceso (mediante `ros2 launch`) junto con `slam_toolbox`, y da la fase por terminada cuando se cumplen a la vez dos condiciones: que se haya superado un tiempo mínimo de exploración (`explore_min_secs`, 900s) y que el robot lleve inactivo un intervalo (`explore_idle_secs`, 180s). La inactividad se mide sobre la odometría `/odom`: cada vez que el desplazamiento supera 5cm se reinicia el contador, de modo que la fase concluye cuando `explore_lite` deja de proponer fronteras y el robot se detiene. En **guardado**, solicita el mapa al servicio `/map_saver/save_map` de forma asíncrona, con umbrales de ocupación `free_thresh = 0,25` y `occupied_thresh = 0,65` en formato *trinary*, y sondea el *future* hasta que la operación concluye. En **planificación**, genera los waypoints a partir de la imagen del mapa (extracto 5.5): con OpenCV umbraliza el espacio libre (píxeles próximos al blanco), erosiona el resultado con un núcleo de 5×5 aplicado dos veces para inflar los obstáculos y separar la ruta de las paredes, y recorre la rejilla resultante (un punto cada 2m, configurable) con un barrido en zig-zag que invierte el sentido en filas alternas; cada celda libre se transforma de píxel a coordenada del mundo, invirtiendo el eje vertical (que en la imagen crece hacia abajo), y se encadena el punto de *dock* al inicio y al

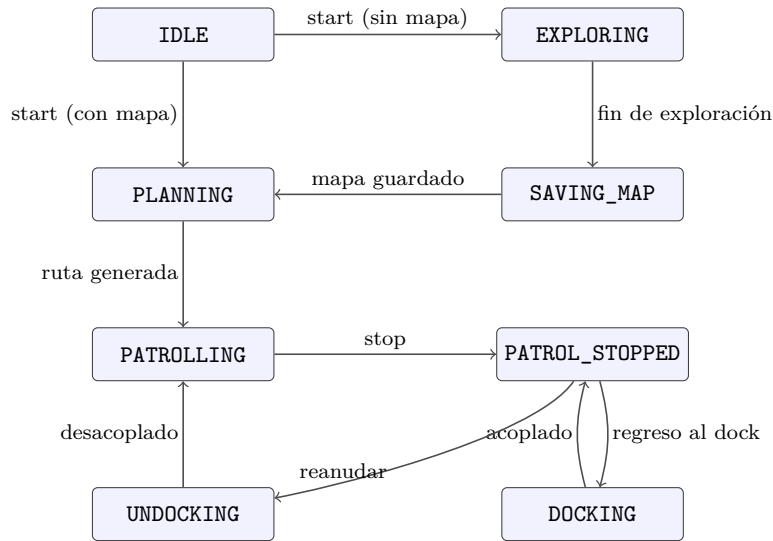


Figura 5.2: Transiciones de la máquina de estados del orquestador `patrol_nav`. Al reanudar, el robot solo pasa por `UNDOCKING` si está acoplado; ese desacople previo se aplica a cualquier arranque que implique conducir (también desde `IDLE`) y, si no hay ruta cargada, `UNDOCKING` puede dar paso a `PLANNING` o `EXPLORING` en lugar de `PATROLLING`. El comando `reinstall` devuelve la FSM a `IDLE` desde cualquier estado. Fuente: elaboración propia.

final. En **patrulla**, recorre cíclicamente la ruta con `goThroughPoses`, reencolando las poses para repetir el recorrido de forma indefinida. La **parada** cancela la navegación en curso y regresa al *dock*; dado que Nav2 procesa las cancelaciones de forma asíncrona, el nodo espera a que la tarea activa concluya (consultando `isTaskComplete`) antes de emitir el objetivo de retorno, evitando que dos órdenes compitan dentro de Nav2.

Una vez alcanzada la base, el robot ejecuta el **acople** con la acción `dock` del *iRobot Create* (estado `DOCKING`) y, al completarse, queda detenido y acoplado; un indicador interno (`stopped_settled`) señala que la secuencia de parada ha terminado, para no repetirla en cada ciclo del temporizador. Por último, la **reinstalación** cancela lo que esté en curso, borra los ficheros del mapa (`.pgm` y `.yaml`) y devuelve la FSM a `IDLE`. El remapeo no puede hacerse en la sesión ya en marcha, pues `explore_lite` requiere SLAM y, en modo de localización, el servidor de mapa sirve un mapa fijo; por ello, tras la reinstalación hay que relanzar el sistema, que, al no encontrar mapa guardado, arrancará de nuevo en SLAM.

Dos decisiones de diseño de esta fase merecen justificación. En primer lugar, los umbrales que determinan el fin de la exploración (`explore_min_secs` y `explore_idle_secs`) se exponen como parámetros de ROS 2 configurables, no como constantes fijas. Sus valores por defecto (900 y 180s) se han escogido de forma deliberadamente conservadora para la validación en simulación, que opera con cierto retraso: la doble condición de tiempo mínimo de exploración e inmovilidad sostenida persigue que una parada prolongada refleje el fin real de la exploración o un fallo, y no un retraso puntual en la mensajería; estos valores deberán recalibrarse al migrar a hardware real. En segundo lugar, la generación de la ruta mediante un barrido en zig-zag sobre la rejilla erosionada es una de las múltiples estrategias posibles de planificación de cobertura (*coverage path planning*) (Galceran & Carreras, 2013); se ha opta-

do por ella priorizando la simplicidad y la robustez frente a la optimalidad, y su sustitución por estrategias más eficientes se contempla como trabajo futuro.

El extracto 5.4 muestra el despacho de la máquina de estados y el manejo del comando de inicio, que reutiliza el mapa si ya existe.

Máquina de estados y arranque (patrol_nav.py)

```

1 def state_machine(self):
2     if self.state == 'IDLE':
3         return # Espera al primer start_patrol del usuario.
4     elif self.state == 'EXPLORING': self.run_exploration()
5     elif self.state == 'SAVING_MAP': self.save_map()
6     elif self.state == 'PLANNING': self.generate_waypoints_from_map()
7     elif self.state == 'PATROLLING': self.run_patrol_cycle()
8     elif self.state == 'PATROL_STOPPED': self.run_patrol_stopped()
9     elif self.state == 'UNDOCKING': self.run_undocking()
10    elif self.state == 'DOCKING': self.run_docking()
11
12 def handle_start_patrol(self):
13     if self.state == 'IDLE':
14         # Reutiliza el mapa si existe; si no, ciclo completo desde cero.
15         self._begin('PLANNING' if self._map_exists() else 'EXPLORING')
16     elif self.state == 'PATROL_STOPPED':
17         self.active_task = None
18         if self.waypoints_queue: # Reanuda donde estaba.
19             self._begin('PATROLLING')
20         elif self._map_exists(): # Replanifica desde el mapa.
21             self._begin('PLANNING')
22         else: # No hay nada: explora.
23             self._begin('EXPLORING')

```

El desacople previo lo gobierna `_begin`: como Nav2 no mueve el robot mientras está acoplado, si lo está el nodo pasa primero por UNDOCKING y retoma después el estado objetivo. Tanto el acople como el desacople se gobiernan con un patrón asíncrono: al correr sobre un *executor* de un solo hilo, cada acción se sondea por pasos en sucesivos ciclos del temporizador, sin bloquear el resto de tareas. La ruta de patrulla, por su parte, se genera recorriendo el espacio libre del mapa en un barrido en zig-zag (extracto 5.5).

Generación de waypoints en zig-zag (patrol_nav.py)

```

1 grid_spacing_pixels = max(1, int(self.grid_spacing_m / resolution))
2 waypoints = []
3 for y in range(0, safe_binary.shape[0], grid_spacing_pixels):
4     x_range = range(0, safe_binary.shape[1], grid_spacing_pixels)
5     if (y // grid_spacing_pixels) % 2 != 0:
6         x_range = reversed(x_range) # Barrido alterno (zig-zag).
7     for x in x_range:
8         if safe_binary[y, x] == 255: # Solo celdas libres seguras.

```

```

9         real_x = origin_x + (x * resolution)
10        real_y = origin_y + ((img.shape[0] - y) * resolution)
11        waypoints.append(self.create_pose(real_x, real_y, 0.0))

```

5.4.2 Detección: detect_incident

El nodo `detect_incident` materializa la percepción del sistema: se suscribe a la imagen en color de la cámara OAK-D y, en cada fotograma, decide si hay una incidencia, entendida como la presencia de una persona (clase `person`) con confianza superior a un umbral configurable. La elección del detector responde a un compromiso entre precisión y latencia: se ha optado por YOLO en su variante *nano* (YOLOv8n), la más ligera de la familia, frente a las variantes *s* o *m*, porque la inferencia se ejecuta de forma continua sobre cada fotograma y el hardware del robot exige un modelo ligero; además, al reducirse la tarea a una decisión binaria, la mayor exactitud de los modelos grandes no resulta determinante. La inferencia corre en el equipo que aloja el nodo, no en la unidad de procesamiento de la propia cámara, y su rendimiento en tiempo real sobre el hardware objetivo aún no se ha caracterizado, al haberse validado el sistema solo en simulación.

El diseño persigue registrar el suceso con su contexto, no solo señalar el instante de la detección. Por eso el nodo mantiene un búfer circular (`deque`) con los últimos cien fotogramas y, al detectar una persona, graba además los cincuenta siguientes, de modo que el clip abarca aproximadamente los cinco segundos previos y los cinco posteriores al disparo. Cerrado el clip, un periodo de enfriamiento de 30s evita que una misma persona genere una cascada de incidencias redundantes. El tamaño del búfer, el número de fotogramas posteriores y la duración del enfriamiento son parámetros ajustables. El extracto 5.6 recoge esta lógica de búfer, disparo y enfriamiento.

Lógica de detección con búfer y enfriamiento (detect_incident.py)

```

1 def image_callback(self, msg):
2     cv_image = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
3     if self.cooldown:
4         return
5     self.incident_buffer.append(cv_image)
6     if self.post_trigger_counter > 0:
7         self.post_trigger_counter -= 1
8         if self.post_trigger_counter == 0:           # Cierra el clip.
9             self.cooldown = True
10            self.timer = self.create_timer(self.cooldown_time,
11                                           ↪ self.reset_cooldown)
12            threading.Thread(target=self.process_incident_io,
13                             args=(list(self.incident_buffer),
14                                   ↪ int(time.time()))),
15                             daemon=True).start()
16 elif self.detect_incident(cv_image):               # Disparo: persona
17     ↪ detectada.
18     self.post_trigger_counter = 50

```


El trabajo pesado se delega en un hilo de entrada/salida independiente, de modo que la captura de imágenes no se interrumpe mientras se procesa la incidencia. Ese hilo codifica el clip en mp4 con OpenCV, lo sube al almacén de objetos MinIO y publica en el bus de eventos un aviso con el identificador del robot, el *bucket* y el nombre del clip, firmado con el JWT del robot. La imagen, por tanto, no viaja por el bus: el evento solo transporta una referencia al vídeo ya almacenado, y el flujo de notificación descrito en la sección 5.1.2 convierte ese aviso en una incidencia consultable desde la aplicación.

5.4.3 Telemetría: `serve_telemetry`

El nodo `serve_telemetry` consolida el estado del robot a partir de seis suscripciones, cada una con su *callback*: batería (`/battery_state`, tipo `BatteryState`), odometría (`/odom`, `Odometry`) y los estados propios de la base *iRobot Create* de atraque (`/dock_status`), secuestro (`/kidnap_status`), deslizamiento (`/slip_status`) y ruedas (`/wheel_status`). Los *callbacks* no publican nada por sí mismos: se limitan a actualizar un diccionario de estado interno (porcentaje y voltaje de batería, posición, velocidades lineal y angular e indicadores booleanos), que actúa como última instantánea conocida. Un temporizador independiente publica ese diccionario consolidado en Kafka una vez por segundo, añadiéndole el `robot_id` y una marca temporal e incorporando el JWT del robot en la cabecera del mensaje; el token se obtiene del servicio `get_robot_token` de la misma forma no bloqueante que en los demás nodos. La tabla 5.6 detalla los campos que componen cada mensaje, con su tipo y la suscripción de la que proceden, y el extracto 5.7 muestra un ejemplo del mensaje JSON publicado.

Estos mensajes alimentan el flujo de telemetría descrito en la sección 5.1.2: el backend los consume y los almacena en InfluxDB, y Grafana los representa en un panel (figura 5.3) que permite seguir el estado del robot en tiempo real.

Ejemplo de mensaje de telemetría (topic `robot_telemetry`)

```
1 {  
2   "robot_id": "turtlebot4-01",  
3   "timestamp": 1751131234.512,  
4   "bateria": 87.5,  
5   "bateria_voltaje": 16.2,  
6   "is_docked": false,  
7   "pos_x": 3.42,  
8   "pos_y": -1.18,  
9   "vel_linear": 0.21,  
10  "vel_angular": 0.05,  
11  "is_kidnapped": false,  
12  "is_slipping": false,  
13  "wheels_enabled": true  
14 }
```

Tabla 5.6: Campos del mensaje de telemetría, con su tipo y la suscripción de origen. Fuente: elaboración propia.

Campo	Tipo	Origen	Descripción
robot_id	cadena	ROBOT_ID	Identificador único del robot (variable de entorno).
timestamp	número	time.time()	Marca temporal Unix del envío, en segundos.
bateria	número	/battery_state	Nivel de batería en porcentaje (0–100).
bateria_voltaje	número	/battery_state	Voltaje de la batería (V).
pos_x	número	/odom	Coordenada X en el mapa (m).
pos_y	número	/odom	Coordenada Y en el mapa (m).
vel_linear	número	/odom	Velocidad lineal (m/s).
vel_angular	número	/odom	Velocidad angular (rad/s).
is_docked	booleano	/dock_status	El robot está atracado en su base.
is_kidnapped	booleano	/kidnap_status	El robot ha sido levantado del suelo.
is_slipping	booleano	/slip_status	Las ruedas patinan sin tracción efectiva.
wheels_enabled	booleano	/wheel_status	Los motores de las ruedas están habilitados.



Figura 5.3: Panel de Grafana que visualiza la telemetría del robot en tiempo real. Fuente: elaboración propia.

5.4.4 Gestión de secretos: secret_helper

El nodo `secret_helper` centraliza la autenticación de todo el espacio de trabajo. Aplica el patrón TOFU (*trust on first use*), cuya premisa es que el robot y el backend establecen una relación de confianza en su primer contacto y la reutilizan en lo sucesivo: en lugar de distribuir

credenciales de antemano, el secreto se acuerda una sola vez, en el arranque inicial, y se da por fiable a partir de entonces. En la práctica, el nodo distingue dos situaciones. Si no existe un secreto local, ejecuta el primer arranque (*first boot*): solicita un secreto al backend con `POST /robot/{id}/first-boot` y lo persiste en la ruta indicada por `SECRET_PATH`, montada como volumen para sobrevivir a las recreaciones del contenedor. Si el secreto ya existe, lo reutiliza para autenticarse con `POST /robot/{id}/auth` y obtener un JWT. Toda esta lógica reside en un hilo en segundo plano (`auth_routine`) que renueva el token de forma proactiva: duerme una hora entre renovaciones, muy por debajo de las veinticuatro horas de validez del JWT, de modo que nunca se utiliza un token caducado; si una autenticación falla, reintenta cada diez segundos. Para que el resto de nodos no tenga que gestionar credenciales, el nodo publica el servicio ROS 2 `get_robot_token`, que devuelve el token vigente junto con un indicador de disponibilidad (`is_ready`); los consumidores lo invocan de forma no bloqueante (mediante `call_async` y un `callback` de finalización) para no detener el `executor` de hilo único, y reintentan si el token todavía no está disponible. Las figuras 5.4 y 5.5 resumen, respectivamente, el arranque inicial con autenticación y la forma en que el resto de nodos obtienen y emplean el token.

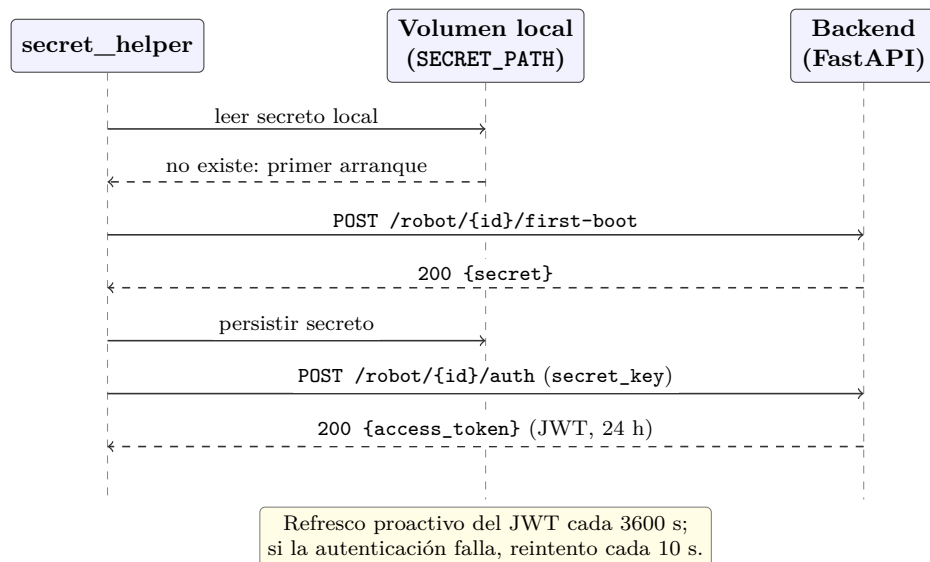


Figura 5.4: Secuencia del arranque inicial (*first boot*, TOFU) y la autenticación de `secret_helper`: si no hay secreto local, lo solicita al backend y lo persiste; después lo intercambia por un JWT que renueva de forma periódica. Fuente: elaboración propia.

Conviene reconocer la principal asunción de este patrón: la confianza depositada en el primer contacto. Si un atacante se adelantara al robot legítimo y completara el `first-boot` antes que él, podría reclamar su identidad. En el contexto de este trabajo el riesgo se considera asumible, ya que el despliegue es local y controlado, el aprovisionamiento del robot se realiza de forma manual en una red de confianza y cada robot ejecuta un único `first-boot`. No obstante, el endurecimiento de este arranque inicial queda como limitación conocida.

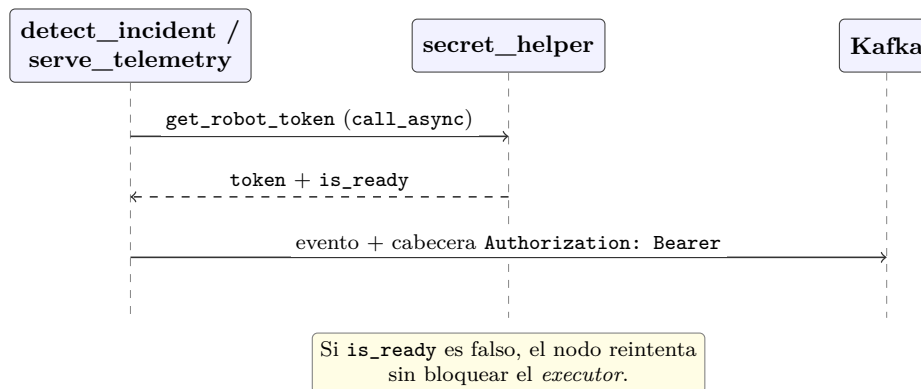


Figura 5.5: Los nodos consumidores obtienen el JWT del servicio `get_robot_token` de `secret_helper` y lo adjuntan en la cabecera `Authorization: Bearer` de los mensajes que publican en Kafka. Fuente: elaboración propia.

5.5 Aplicación de usuario

La aplicación con la que interactúa el usuario está desarrollada en *Flutter*, lo que permite desplegarla en múltiples plataformas (móvil, web y escritorio) desde un único código. Esta sección describe primero su arquitectura interna y, a continuación, recorre las pantallas que la componen, ilustradas con capturas de la aplicación en funcionamiento.

5.5.1 Arquitectura de la aplicación

La aplicación separa la presentación, el estado y el acceso a datos. La comunicación con la API REST se concentra en una capa de repositorios, uno por dominio (autenticación, robots, incidencias, usuario y administración), construida sobre el cliente *dio*, que toma la dirección base del backend de `AppConfig.baseUrl` y adjunta el JWT de la sesión a cada petición. El estado se gestiona con *Riverpod*: controladores para el estado mutable (sesión, robots) y *providers* de tipo *future* para las lecturas puntuales (perfil, incidencias, listados de administración). Cada pantalla consume ese estado a través del patrón `AsyncValue`, que distingue de forma uniforme los tres casos posibles (cargando, con datos o con error) y se traduce en los componentes compartidos `LoadingView` y `ErrorView` (este último con reintento), de modo que el tratamiento de la carga y de los fallos es homogéneo en toda la aplicación.

La navegación se gestiona con `go_router`, cuya función de redirección centraliza el control de acceso en el cliente: si no hay sesión válida, cualquier ruta se redirige a la pantalla de acceso; una vez autenticado, esa pantalla reenvía a la de inicio; y la ruta de administración queda reservada a los usuarios con rol de administrador. El extracto 5.8 muestra esa lógica. El menú lateral (figura 5.6), común a las pantallas principales, da acceso a las secciones y a la opción de cerrar sesión, e incorpora el conmutador de tema claro y oscuro.

Control de acceso en el router (router.dart)

```

1 redirect: (context, state) {
2   final isAuthRoute = state.matchedLocation == '/auth';
3   // Sin sesion valida: solo se permite la pantalla de acceso.
4   if (auth.status != AuthStatus.authenticated) {
5     return isAuthRoute ? null : '/auth';
6   }
7   if (isAuthRoute) return '/home';           // Con sesion: ir a Inicio.
8   // Administracion: requiere rol de administrador.
9   if (state.matchedLocation == '/admin' && auth.role != 'admin') {
10    return '/home';
11  }
12  return null;
13 }

```

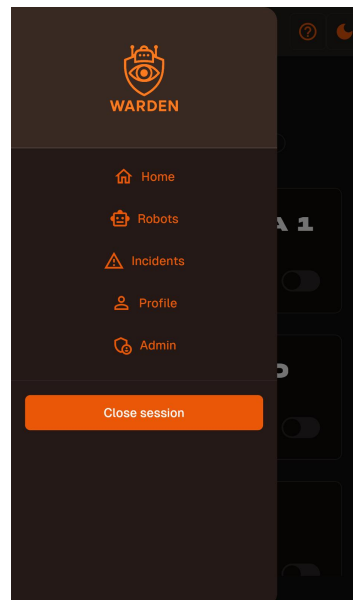


Figura 5.6: Menú lateral de navegación, con acceso a las secciones y al cierre de sesión. Fuente: elaboración propia.

5.5.2 Recorrido por las pantallas

La tabla 5.7 relaciona cada pantalla con su ruta y con los principales endpoints del backend (tabla 5.3) que consume. A continuación se describe cada una.

Acceso. La pantalla de acceso (figura 5.7) ofrece, en dos pestañas, el inicio de sesión (correo y contraseña) y el registro de un nuevo usuario (nombre, correo, contraseña y teléfono opcional). Un registro correcto inicia sesión de forma automática, de modo que el usuario accede directamente a la aplicación.

Tabla 5.7: Pantallas de la aplicación, su ruta y los endpoints que consumen. Fuente: elaboración propia.

Pantalla	Ruta	Endpoints principales
Acceso	/auth	POST /user/register, /user/login
Inicio	/home	GET /user/robots; PUT /user/{id}/start-patrol, stop-patrol
Robots	/robots	GET /user/robots, dashboard-url; POST /user/new-robot; PUT .../alias, reinstall; DELETE .../delete
Incidencias	/incidents	GET /user/incidents
Detalle	/incidents/:id	GET /user/incidents/{id}/show-video
Perfil	/profile	GET /user/profile; PUT /user/update-info, update-password
Administración	/admin	GET /admin/users, /admin/robots; PUT .../grant, revoke; DELETE .../delete

Inicio. La pantalla principal, «WARDEN» (figura 5.8), lista los robots del usuario con su estado de conexión y operativo, y permite iniciar o detener la patrulla de cada uno mediante un conmutador. Un indicador general resume si el conjunto está desarmado o asegurado. La pantalla sondea el estado real cada cinco segundos para reflejar el *heartbeat* del robot e impide detener un robot que todavía se está configurando (mapeando), mostrando un aviso si se intenta. Ofrece además ayuda contextual y el conmutador de tema claro y oscuro.

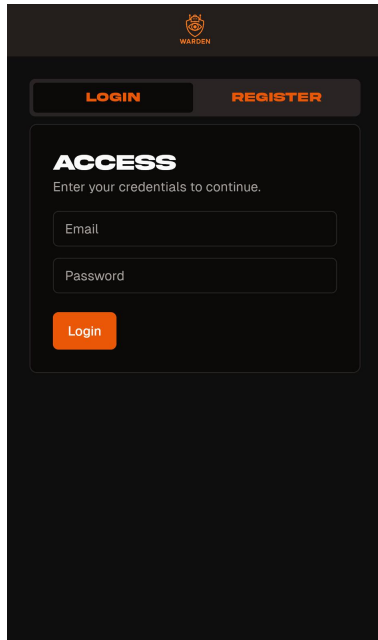
Robots. La pantalla de robots (figura 5.9) gestiona el parque de robots del usuario. Cada robot permite editar su alias, reinstalarlo (lo que borra su mapa y reinicia el ciclo, con confirmación previa), abrir su panel de telemetría en Grafana y eliminarlo. El alta de un robot se realiza mediante un diálogo que solicita su identificador y un alias.

Incidencias y detalle. El listado de incidencias (figura 5.10) muestra cada suceso con un identificador abreviado, el robot que lo originó, la fecha y su estado (revisado o pendiente). Al abrir el detalle, la aplicación reproduce el clip de vídeo asociado mediante un reproductor incrustado que carga la URL prefirmada de MinIO, con repliegue a la apertura en el navegador si la reproducción falla. El detalle ofrece además un acceso directo para avisar a las autoridades.

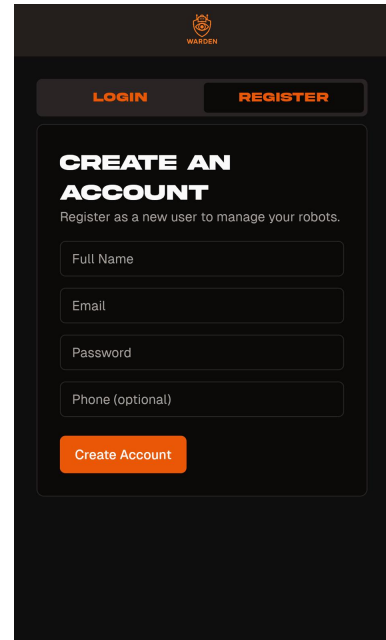
Perfil. La pantalla de perfil (figura 5.11) permite consultar y editar los datos personales del usuario, así como cambiar la contraseña, en dos formularios independientes.

Administración. Disponible solo para administradores, el panel de administración (figura 5.12) se organiza en tres pestañas: usuarios, donde se conceden o revocan privilegios de administrador y se eliminan cuentas; robots, donde se listan y eliminan los robots del sistema; e incidencias, donde se consultan las de un robot concreto a partir de su identificador.

Las figuras 5.7 a 5.12 recogen las capturas de las pantallas descritas, en el mismo orden en que se han presentado.

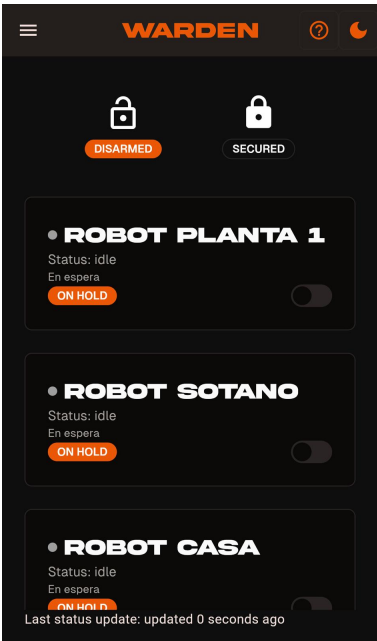


(a) Inicio de sesión.

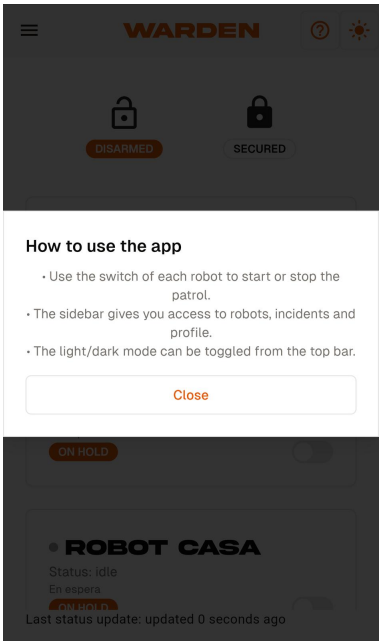


(b) Registro de usuario.

Figura 5.7: Pantalla de acceso, con las pestañas de inicio de sesión y registro. Fuente: elaboración propia.

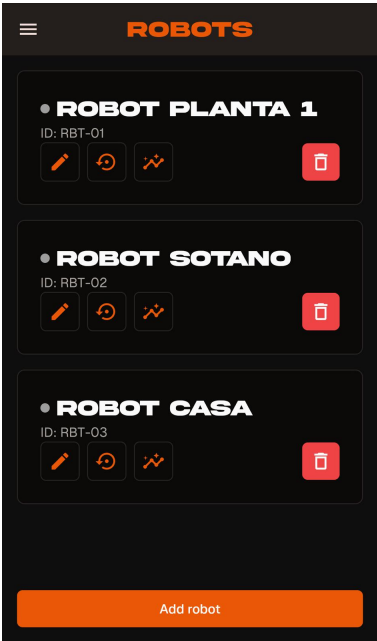


(a) Lista de robots y estado.

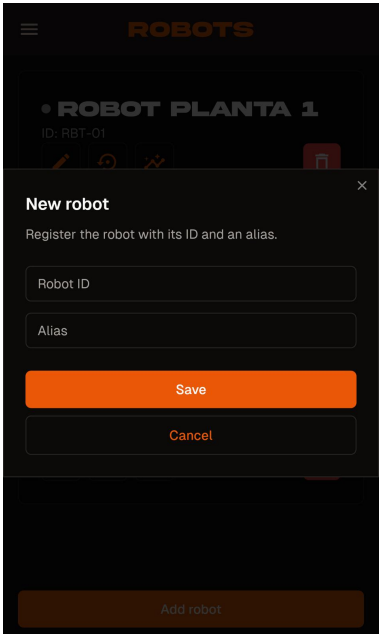


(b) Ayuda contextual (tema claro).

Figura 5.8: Pantalla de inicio: lista de robots con control de patrulla (a) y ayuda contextual en tema claro (b). Fuente: elaboración propia.

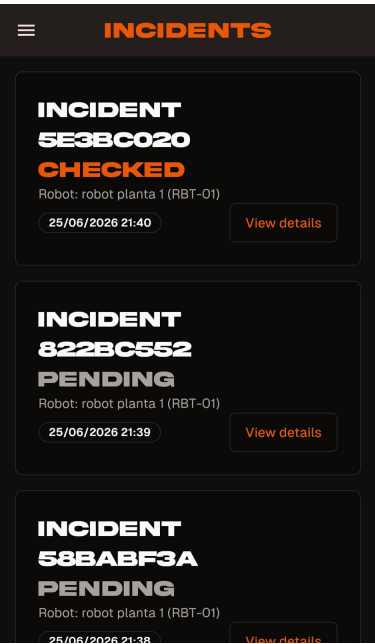


(a) Parque de robots.

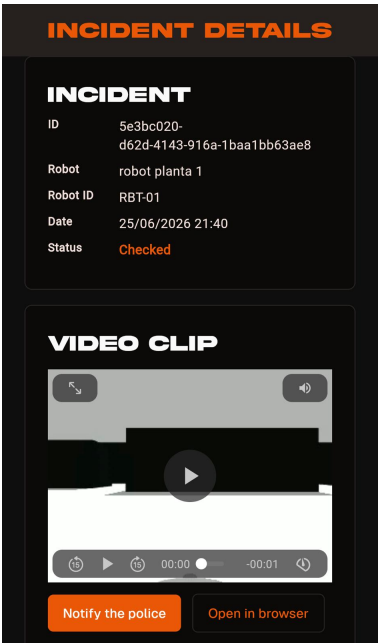


(b) Alta de un robot.

Figura 5.9: Gestión del parque de robots (a) y diálogo de alta (b). Fuente: elaboración propia.



(a) Listado de incidencias.



(b) Detalle con reproducción del vídeo.

Figura 5.10: Listado de incidencias (a) y detalle con el clip de vídeo (b). Fuente: elaboración propia.

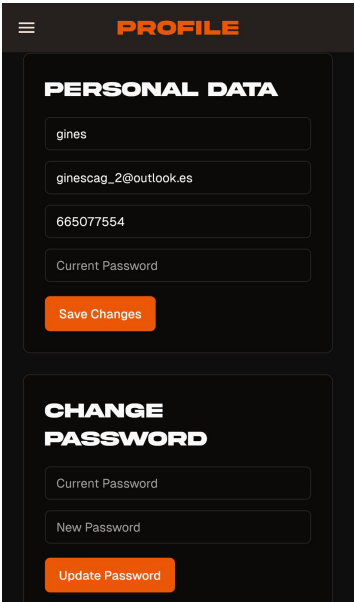


Figura 5.11: Pantalla de perfil, con la edición de datos personales y el cambio de contraseña. Fuente: elaboración propia.

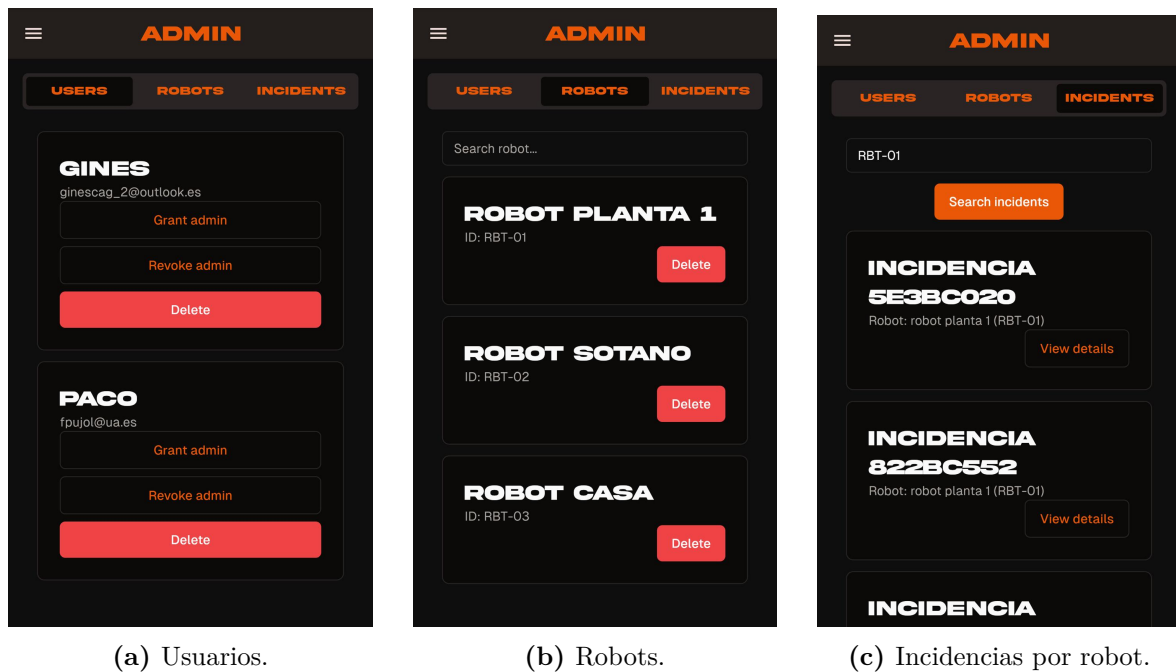


Figura 5.12: Panel de administración: gestión de usuarios (a), robots (b) e incidencias por robot (c). Fuente: elaboración propia.

Con la descripción de la aplicación de usuario se completa el recorrido por las capas del sistema, desde la infraestructura hasta la interfaz con la que opera el usuario. El Capítulo 6 aborda la validación del conjunto: el alcance de las pruebas realizadas, el escenario previsto y el estado actual de la evaluación.

6 Resultados

Este capítulo presenta los resultados del trabajo. El grueso de la evaluación se ha realizado en un entorno de simulación que reproduce la pila de software completa del robot y permite ejercitar el sistema de extremo a extremo. Sobre ese entorno se aporta una doble evidencia: una **validación funcional**, que comprueba que el ciclo operativo completo se ejecuta de principio a fin, y un primer conjunto de **métricas cuantitativas** de cartografía, navegación, detección y rendimiento. A lo largo del capítulo se distingue de forma deliberada entre lo que se ha **implementado**, lo que se ha **validado en simulación** y lo que se ha **probado sobre hardware real**, comentando también las diferencias y limitaciones encontradas. En esa última línea, la Sección 6.4 recoge una campaña de testing sobre el TurtleBot 4 físico que valida en hardware real varios de los subsistemas y documenta, cuantificándolas, las limitaciones halladas en el despliegue.

6.1 Entorno y escenario de pruebas

La evaluación se ha llevado a cabo sobre el modelo del TurtleBot 4 ejecutado en *Gazebo*, con *RViz* como herramienta de visualización del estado del robot, los sensores y el mapa. El mundo simulado es un entorno interior de tipo laberinto, con varios corredores y bifurcaciones, que ejercita la navegación a través de pasillos estrechos y giros. Este entorno integra la misma pila de software que se ejecutaría sobre el robot físico (ROS 2 Humble, Nav2, `slam_toolbox` y `explore_lite`), de modo que los nodos propios del proyecto y el backend operan sin cambios respecto al despliegue real. Esta equivalencia permite ejercitar la lógica completa del sistema (la máquina de estados del orquestador, la generación de la ruta, la detección y los flujos de datos) sin depender todavía del hardware.

La descripción anterior corresponde al entorno de simulación. De forma complementaria, se llevó a cabo una campaña de pruebas sobre el robot físico en un laboratorio interior, cuyo escenario, metodología y resultados, incluidas las limitaciones detectadas en el despliegue, se desarrollan con detalle en la Sección 6.4.

6.2 Validación funcional

En primer lugar, la validación se ha orientado a verificar la correcta integración de los subsistemas y la coherencia de los flujos de datos entre el robot, el bus de eventos, el backend y la aplicación de usuario. Sobre el entorno de simulación se ha ejercitado el ciclo operativo completo, comprobando que cada flujo descrito en el Capítulo 5 se ejecuta de principio a fin y produce el efecto esperado en los almacenes de datos y en la interfaz.

En particular, se ha confirmado que el robot explora el entorno y construye su mapa, que la FSM encadena de forma autónoma las fases del ciclo (exploración, guardado, planificación y patrulla), que la ruta de cobertura se deriva del mapa y se recorre de forma cíclica, y

que la detección de una persona dispara el registro de una incidencia con su vídeo asociado. Asimismo, se ha verificado que la telemetría llega a InfluxDB y se visualiza en Grafana, y que las órdenes emitidas desde la aplicación (iniciar, detener y reinstalar la patrulla) se propagan hasta el robot y se reflejan en su estado. La tabla 6.1 resume estas comprobaciones.

Tabla 6.1: Resumen de la validación funcional realizada en simulación. Fuente: elaboración propia.

Flujo o prueba	Cobertura	Resultado
Autenticación del robot	TOFU, autenticación y refresco del JWT	Verificado (sim.)
Exploración y mapeo	<code>explore_lite</code> y <code>slam_toolbox</code>	Verificado (sim.)
Guardado del mapa	Servicio <code>map_saver</code>	Verificado (sim.)
Generación de la ruta	Barrido en zig-zag sobre el mapa	Verificado (sim.)
Patrulla cíclica	Recorrido de <i>waypoints</i> con Nav2	Verificado (sim.)
Acople y desacople	Acciones <code>dock/undock</code> del Create 3	Verificado (sim.)
Localización sobre mapa guardado	<code>map_server</code> y AMCL (modo localización)	Verificado (sim.)
Detección de incidencia	YOLO → MinIO → Kafka → registro	Verificado (sim.)
Telemetría	<code>serve_telemetry</code> → InfluxDB → Grafana	Verificado (sim.)
Control desde la aplicación	Iniciar, detener y reinstalar la patrulla	Verificado (sim.)
Estado operativo	<i>Heartbeat</i> → PostgreSQL → app	Verificado (sim.)

Estas comprobaciones confirman que la arquitectura es coherente y que los subsistemas se integran según el diseño. A partir de ellas, las secciones siguientes cuantifican el comportamiento del sistema en simulación.

6.3 Resultados de simulación

Más allá de la validación funcional, sobre el mismo entorno de simulación se ha recogido un primer conjunto de métricas cuantitativas, organizadas según las dimensiones definidas en la Sección 4.3: cartografía y navegación, detección de incidencias y rendimiento de extremo a extremo.

6.3.1 Cartografía y ruta de patrulla

El robot explora el entorno de forma autónoma con `explore_lite` mientras `slam_toolbox` construye el mapa de ocupación (figura 6.1a). A partir de ese mapa, el orquestador deriva la ruta de cobertura en zig-zag y la recorre de forma cíclica; la figura 6.1b muestra la trayectoria seguida sobre el mapa ya consolidado.

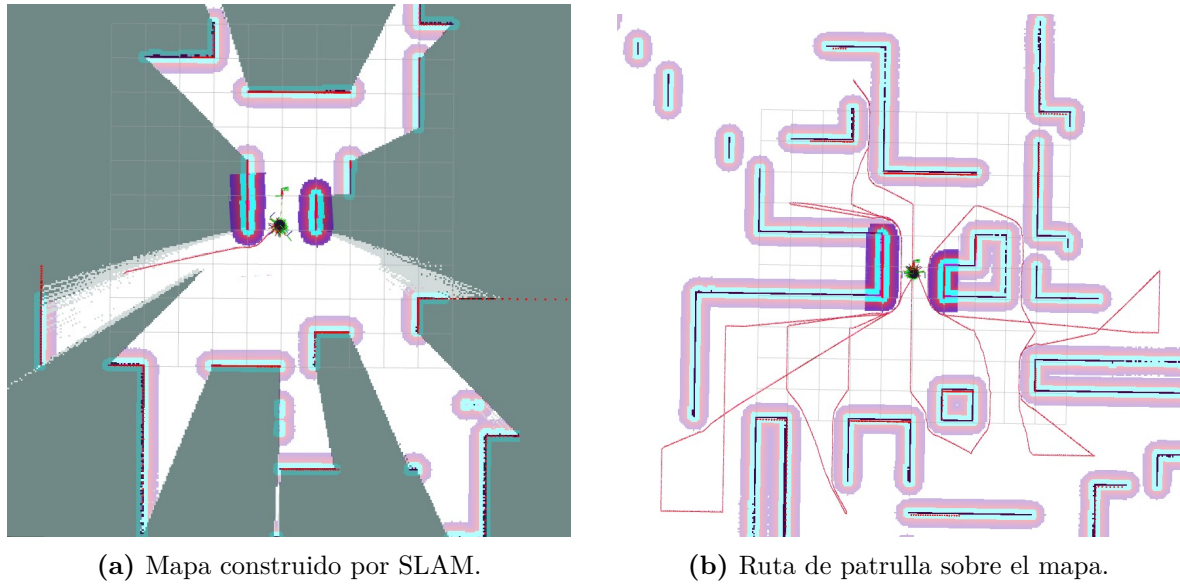


Figura 6.1: Cartografía y patrulla en simulación (RViz): (a) mapa de ocupación construido por `slam_toolbox` durante la exploración (blanco: espacio libre; rosa y morado: paredes y su inflado; gris: sin explorar); (b) trayectoria de la patrulla (en rojo) sobre el mapa consolidado, derivada del barrido en zig-zag del espacio libre. Fuente: elaboración propia.

El mapa resultante, guardado con resolución de 0,05 m por celda, cubre un lienzo de 390×401 celdas (unos $19,5 \times 20,1$ m). De la región ya explorada, en torno al 96,5 % de las celdas corresponde a espacio libre y el 3,5 % restante a paredes, lo que arroja un área libre navegable de aproximadamente $185,5 \text{ m}^2$. La tabla 6.2 recoge estas magnitudes junto con las métricas de navegación que completan la corrida de patrulla.

Tabla 6.2: Métricas de cartografía y navegación en simulación. Fuente: elaboración propia.

Métrica	Unidad	Valor
Resolución del mapa	m/celda	0,05
Extensión cartografiada (lienzo)	m	$19,5 \times 20,1$
Área libre navegable	m^2	$\approx 185,5$
Paredes sobre la región explorada	%	3,5
Tiempo de exploración del mapa (aproximado)	min	≈ 14
Waypoints generados en la ruta	n°	29
Waypoints alcanzados sin intervención	n°	29 / 29
Comportamientos de recuperación de Nav2	n°	2
Colisiones durante la patrulla	n°	0
Tiempo por ciclo de patrulla (aproximado)	min	≈ 20

■ Inciso sobre la obtención de métricas en simulación Las métricas de la tabla 6.2 de carácter temporal resultan afectadas por la velocidad de simulación, que depende de la

carga de la máquina y de la complejidad del mundo virtual.

6.3.2 Detección de incidencias

La capacidad de detección se evalúa sobre un escenario controlado construido en simulación, diseñado para poder contrastar cada detección con la verdad conocida del entorno. Sobre el mundo interior descrito en la Sección 6.1 se introdujeron, en poses e instantes conocidos, modelos tridimensionales realistas de persona junto con objetos de otra naturaleza, entre ellos un maniquí de apariencia humana incluido de forma deliberada como el caso más ambiguo y exigente, con la finalidad de que el robot navegara a través del entorno encontrándose con posibles incidencias. En total se registraron 162 apariciones: 120 de personas (positivos reales) y 42 de no-personas (negativos reales).

Durante estas apariciones se ejecutó el **nodo de producción** `detect_incident`, sin modificación alguna respecto al despliegue real: aplica el modelo YOLOv8n a cada fotograma de la cámara frontal y, cuando la confianza de una detección de la clase `person` supera el umbral fijado en 0,5, dispara la incidencia. En paralelo, y sin interferir con ese nodo, se empleó un visor independiente que ejecuta el mismo modelo sobre el mismo tópico de cámara para registrar la confianza de cada detección y el tiempo de inferencia por fotograma. La figura 6.2 muestra una de estas detecciones durante la patrulla.

Tomando el guion de apariciones como referencia, cada una se cotejó con la respuesta del detector y se clasificó como verdadero positivo, falso negativo o falso positivo, conforme a las definiciones de la Sección 4.3; de ese recuento se derivan la precisión (verdaderos positivos sobre el total de detecciones positivas) y el *recall* (verdaderos positivos sobre el total de apariciones de persona). La tabla 6.3 recoge estos valores junto con la confianza media y el tiempo de inferencia. Conviene subrayar que se trata de un escenario controlado en simulación: las poses conocidas, los modelos realistas de persona y el umbral fijo propician condiciones nítidas, de modo que estas cifras acotan el comportamiento del detector en ese entorno y no sustituyen a su evaluación con personas reales, reservada para la campaña sobre hardware real (Sección 6.4).



Figura 6.2: Detección de una persona durante la patrulla en simulación, que dispara el registro de la incidencia. Fuente: elaboración propia.

Tabla 6.3: Métricas de detección sobre el escenario controlado en simulación. Fuente: elaboración propia.

Métrica	Unidad	Valor
Apariciones evaluadas	nº	162
de persona (positivos reales)	nº	120
de no-persona (negativos reales)	nº	42
Verdaderos positivos	nº	120
Falsos negativos	nº	0
Falsos positivos	nº	7
Verdaderos negativos	nº	35
Precisión	[0, 1]	0,945
<i>Recall</i>	[0, 1]	1,000
Confianza media de detección	[0, 1]	0,899
Tiempo de inferencia	ms/fotograma	80,3

El detector se enfrentó a 162 apariciones controladas durante la patrulla, de las cuales 120 correspondían a personas y 42 a objetos de otra naturaleza. Sobre las 120 apariciones de persona el modelo acertó en todas, sin un solo falso negativo, y lo hizo con una confianza media de 0,899, lo que indica detecciones no solo correctas sino también firmes. El *recall* resultante es, por tanto, perfecto (1,000).

Frente a las 42 apariciones que no eran personas, el modelo las descartó correctamente en 35 ocasiones y se equivocó en 7, todas ellas sobre un maniquí de apariencia humana, el caso más exigente del conjunto. Esos siete falsos positivos son la única fuente de error y rebajan la precisión a 0,945, un valor todavía elevado y coherente con un sistema cuya prioridad es no pasar por alto a ninguna persona. El tiempo medio de inferencia fue de 80,3 ms por

fotograma; conviene precisar que se midió sobre la CPU del equipo de simulación y no sobre el hardware del robot, por lo que constituye una referencia en ese entorno y no el rendimiento esperable a bordo.

Merece la pena detenerse en el umbral de confianza escogido. Se fijó de forma deliberada en un valor relativamente bajo, 0,5, porque en un sistema de vigilancia el error más costoso es el falso negativo, esto es, no detectar a una persona que sí está presente, en especial en los casos límite (encuadres parciales, poca iluminación o posturas poco habituales). Un umbral bajo desplaza esa incertidumbre hacia el lado seguro, al precio de admitir más falsos positivos, y esa es precisamente la razón de los siete disparos sobre el maniquí. La decisión asume ese coste porque resulta aceptable lanzar una incidencia que después se revele intrascendente, mientras que pasar por alto una presencia real comprometería el propósito del sistema. En otras palabras, se prioriza el *recall* frente a la precisión, en coherencia con la finalidad de vigilancia de la aplicación.

6.3.3 Validación del flujo de extremo a extremo

Por último, se valida el flujo completo de notificación de principio a fin. En simulación se ha comprobado que una incidencia detectada por el robot recorre toda la cadena sin intervención manual: el nodo `detect_incident` publica el evento en Kafka, el consumidor del backend lo registra como un `Incidente` en PostgreSQL y la aplicación lo lista y reproduce el clip almacenado en MinIO. De forma análoga, la telemetría que el robot publica una vez por segundo llega a InfluxDB y se visualiza en Grafana (figura 5.3), y el *heartbeat* de estado, emitido cada diez segundos, actualiza el estado operativo que la aplicación refleja en la pantalla de inicio. El sistema funciona, por tanto, de extremo a extremo en simulación.

La latencia absoluta percibida, desde que la persona aparece hasta que la incidencia es visible en la aplicación, no se reporta en este entorno: el simulador opera por debajo del tiempo real y el clip incorpora además un tramo de fotogramas posteriores grabados a la cadencia reducida de la cámara simulada, de modo que cualquier medida en reloj de pared resultaría inflada y no representativa. Su caracterización se reserva para la campaña sobre hardware real (sección 6.4). Sí se recogen, en cambio, las magnitudes cuyo valor no depende del factor de tiempo real de la simulación, por ejecutarse en el anfitrión o por referirse a la fiabilidad del flujo, que la tabla 6.4 resume.

Tabla 6.4: Métricas del flujo de extremo a extremo medibles en simulación, independientes del factor de tiempo real. Fuente: elaboración propia.

Métrica	Unidad	Valor
Telemetría recibida frente a la esperada (1 Hz nominal)	%	100
Latencia evento en Kafka → registro en PostgreSQL	ms	503
Tiempo de subida del clip a MinIO (media de 20 incidencias)	s	3,7

Las magnitudes recogidas confirman la solidez del flujo en el lado del servidor, cuyo rendimiento no depende de la velocidad de la simulación. La telemetría se recibió sin pérdidas,

con todos los puntos esperados a la cadencia nominal de un punto por segundo. La subida de cada clip de incidencia a MinIO tardó 3,7s de media sobre veinte incidencias, un tiempo dominado por la escritura y la transferencia del vídeo. Por último, la latencia desde la llegada del evento a Kafka hasta su registro en PostgreSQL fue de 503ms; este valor está dominado por el ciclo de sondeo del consumidor, que espera los mensajes en intervalos de hasta un segundo (`poll(1.0)`), más que por el procesamiento o la escritura en la base de datos, y podría reducirse acortando ese intervalo.

6.4 Pruebas en el robot real

Además de la evaluación en simulación, se llevó a cabo una campaña de pruebas sobre el TurtleBot 4 físico en un laboratorio interior. Esta campaña persigue un doble objetivo: comprobar que la pila de software del proyecto opera sin cambios contra el hardware real y detectar aquellos aspectos que la simulación no revela, como la dinámica de los sensores reales, la sincronización de relojes entre dispositivos y el comportamiento de la red de despliegue bajo carga. Los resultados son de carácter *preliminar*: varios subsistemas quedaron validados de extremo a extremo sobre el robot, mientras que la validación completa del ciclo de patrulla y de la detección se vio limitada por dos factores ajenos al software desarrollado, la estabilidad del transporte de datos en el banco de pruebas y un fallo de la cámara, que se documentan y cuantifican a continuación.

6.4.1 Entorno y metodología

El robot es un TurtleBot 4 (base iRobot Create 3, Raspberry Pi 4, LiDAR RPLIDAR y cámara OAK-D-LITE) conectado a una red wifi aislada. El equipo de desarrollo es un portátil con Windows 10 que, por las limitaciones de su subsistema de virtualización de red para el descubrimiento DDS, ejecuta el software del proyecto dentro de una máquina virtual (VirtualBox, Ubuntu 22.04) configurada con adaptador de red en modo puente sobre la interfaz wifi. De este modo la máquina virtual obtiene una dirección propia en la subred del robot y los contenedores ROS 2 de la pila arrancan con la red del anfitrión, condición necesaria para el tráfico DDS bidireccional.

La metodología de cada sesión consistió en levantar la infraestructura de *backend* (Kafka, MinIO, PostgreSQL, InfluxDB, Grafana y la API) en la máquina virtual, arrancar el servidor de descubrimiento y el *bringup* de navegación contra el robot, lanzar los nodos del proyecto y, finalmente, emitir la orden de inicio de patrulla desde la aplicación a través de Kafka. La observación se apoyó en cuatro fuentes simultáneas: los registros de los nodos, la visualización en RViz del mapa y los sensores, los cuadros de mando de Grafana y la propia aplicación de usuario.

Conviene precisar la naturaleza de este banco de pruebas, porque condiciona los resultados. La máquina virtual conectada sobre wifi introduce una capa de virtualización y un enlace inalámbrico entre los nodos de cómputo y el robot. Como se verá, esa capa condiciona el rendimiento del transporte bajo carga y constituye una limitación para la campaña de pruebas, no atribuible al software del proyecto.

6.4.2 Sincronización temporal: los tres relojes

El primer obstáculo específico del hardware real fue la sincronización horaria. A diferencia de la simulación, donde existe un único reloj, en el robot real intervienen **tres relojes independientes**: la Raspberry Pi (que marca la hora de los mensajes del LiDAR), la base Create 3 (que marca la odometría y las transformadas) y la máquina virtual. Cualquier desfase entre ellos provoca que `slam_toolbox` descarte los escaneos por llegar con una marca temporal anterior a los datos disponibles en la caché de transformadas, con el consiguiente colapso del mapeo. Un agravante adicional es que las *Guest Additions* de VirtualBox reimponen periódicamente la hora del anfitrión sobre la máquina virtual, deshaciendo el trabajo de sincronización.

La solución adoptada combina tres medidas: sincronizar la máquina virtual contra la Raspberry Pi mediante `chrony`; deshabilitar la sincronización horaria de VirtualBox para que no compita con `chrony`; y, cuando el desfase de la base Create 3 es grande, forzar su resincronización por NTP o reiniciarla. La verificación se realizó comparando las marcas de tiempo justo antes de cada prueba: la diferencia entre la Raspberry Pi y la máquina virtual quedó en el orden de unos pocos milisegundos, y la de la base Create 3 en cero segundos.

```

title=Verificación de la sincronización de los tres relojes

$ ssh ubuntu@192.168.31.191 'date +%s.%N'; date +%s.%N
1783088693.619321844 # Raspberry Pi
1783088693.626592666 # maquina virtual (diferencia ~7 ms)

$ ros2 topic echo /turtlebot4/odom --field header.stamp --once; date +%s
sec: 1783088743 # marca temporal de la base Create 3 (odometria)
1783088743 # reloj de la maquina virtual (diferencia 0 s)

```

6.4.3 Subsistemas validados en hardware real

Sobre el robot físico quedaron validados de extremo a extremo la autenticación, la telemetría y la cadena de mando del orquestador.

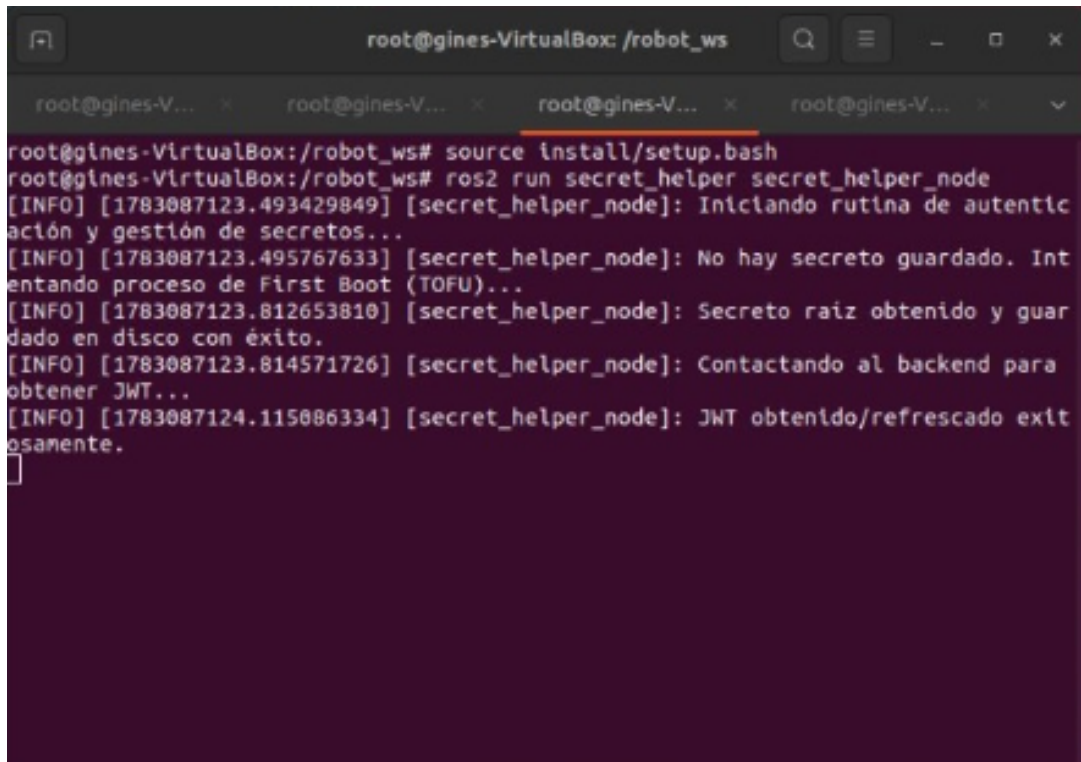
Autenticación (`secret_helper`). El nodo completó el proceso de TOFU contra el *backend* real: obtuvo y persistió el secreto raíz en el primer arranque y canjeó a continuación el JWT. La figura 6.3 muestra la traza simultánea del nodo y del *backend*, donde se observan las peticiones `first-boot` y `auth` resueltas con código 200.

```

title=Traza del nodo secret_helper en el primer arranque real

[secret_helper_node]: Iniciando rutina de autenticacion y gestion de
secretos...
[secret_helper_node]: No hay secreto guardado. Intentando proceso de First Boot
(TOFU)...
[secret_helper_node]: Secreto raiz obtenido y guardado en disco con exito.
[secret_helper_node]: Contactando al backend para obtener JWT...
[secret_helper_node]: JWT obtenido/refrescado exitosamente.

```



```
root@gines-VirtualBox: /robot_ws
root@gines-VirtualBox: /robot_ws# source install/setup.bash
root@gines-VirtualBox: /robot_ws# ros2 run secret_helper secret_helper_node
[INFO] [1783087123.493429849] [secret_helper_node]: Iniciando rutina de autenticación y gestión de secretos...
[INFO] [1783087123.495767633] [secret_helper_node]: No hay secreto guardado. Iniciando proceso de First Boot (TOFU)...
[INFO] [1783087123.812653810] [secret_helper_node]: Secreto raíz obtenido y guardado en disco con éxito.
[INFO] [1783087123.814571726] [secret_helper_node]: Contactando al backend para obtener JWT...
[INFO] [1783087124.115086334] [secret_helper_node]: JWT obtenido/refrescado exitosamente.
```

Figura 6.3: Autenticación del robot real: el nodo `secret_helper` ejecuta el TOFU, obtiene el secreto raíz y canjea el JWT; el *backend* registra las peticiones `first-boot` y `auth`. Fuente: elaboración propia.

Telemetría (`serve_telemetry`). La telemetría del robot (batería, voltaje, odometría y estado de acople, entre otros) fluyó desde el robot hasta InfluxDB y se visualizó en el cuadro de mando de Grafana (figura 6.4). Para ejercitarla se teleoperó el robot con el paquete `teleop_twist_keyboard`, de modo que el movimiento real generado se reflejara en las gráficas de velocidad lineal y angular y de posición. La continuidad de todas las series, sin cortes ni huecos, confirma que no se perdió ninguna muestra entre el robot y el *backend* a la cadencia nominal de un punto por segundo. El cuadro de mando permite además leer el comportamiento físico del robot: los picos del voltaje de batería coinciden con los acoples y desacoples del *dock*, cuando la base pasa a recibir corriente de carga, y los indicadores booleanos de estado (rueda activa, acople y secuestro, entre otros) reflejaron correctamente cada situación. Esta evidencia confirma la cadena robot $\rightarrow$ bus de eventos $\rightarrow$ *backend* $\rightarrow$ visualización sobre hardware real.

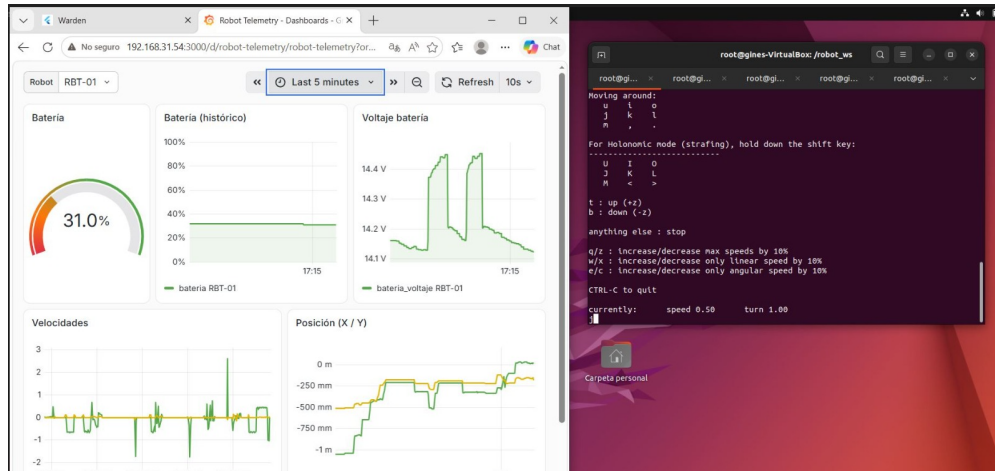


Figura 6.4: Cuadro de mando de Grafana con la telemetría real del TurtleBot 4 (batería, voltaje de batería, velocidades lineal y angular y posición), alimentado por el nodo `serve_telemetry` a través de Kafka e InfluxDB. Fuente: elaboración propia.

6.4.4 Navegación y patrulla: validación parcial

La lógica de integración de la navegación quedó demostrada sobre el hardware. El *bringup* completó su arranque contra el robot real: `slam_toolbox` registró el LiDAR, Nav2 activó sus servidores hasta el mensaje `Nav2 is ready for use!`, y la orden `start_patrol` emitida por Kafka desde la aplicación fue consumida por el orquestador, cuya FSM transitó del estado de reposo al de exploración lanzando el nodo `explore_lite`. Las acciones de acople y desacople del Create 3 respondieron correctamente. Con Nav2 activo, el robot inició la construcción del mapa del laboratorio y este creció de forma efectiva, cubriendo una estancia de aproximadamente $7,9 \times 9,2$ m. Durante esta fase, la aplicación de usuario reflejó en tiempo real el estado de exploración del robot, con RViz y el orquestador `patrol_nav` en marcha (figura 6.5).

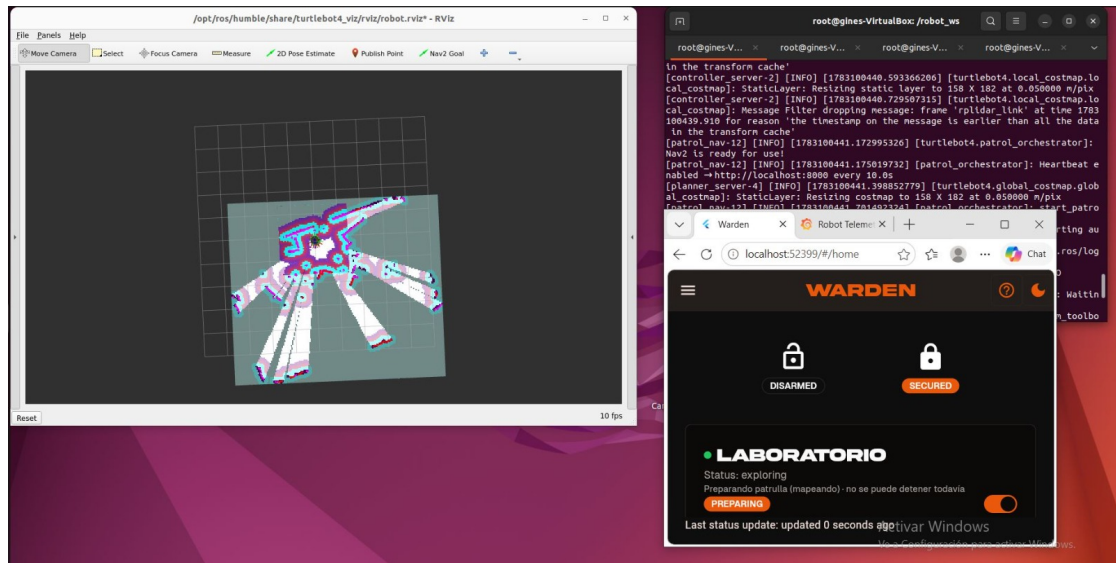


Figura 6.5: Exploración en curso sobre el robot físico: la aplicación de usuario refleja el estado «explorando / preparando patrulla», RViz visualiza el mapa parcial del laboratorio (los artefactos radiales son consecuencia de la entrega degradada del escáner láser bajo carga de red, véase el texto) y la terminal muestra el orquestador `patrol_nav` y Nav2 en ejecución. Fuente: elaboración propia.

No obstante, el ciclo de patrulla no llegó a cerrarse. El mapa crecía de forma intermitente y con artefactos, y la exploración nunca alcanzó su condición de fin. Para localizar la causa se procedió por descarte: se verificó que no era el desfase de relojes (comprobado a cero segundos justo antes de la prueba), ni la saturación de Central Processing Unit (CPU) (medida en torno al 89 % de inactividad), ni la configuración del middleware (las transformadas y los mensajes pequeños fluían con normalidad y Nav2 llegó a activarse). Descartadas esas capas, se midió la entrega del escáner láser (`/scan`) en la máquina virtual bajo dos condiciones: con el robot publicando en reposo y con la pila de navegación completa en marcha. El contraste, recogido en la tabla 6.5, es concluyente.

Tabla 6.5: Degradación de la entrega del escáner láser (`/scan`) en la máquina virtual, medida en reposo frente a bajo carga de la pila de navegación (n. d.: no medido de forma fiable bajo carga). Fuente: elaboración propia.

Métrica	En reposo	Bajo carga
Frecuencia media de <code>/scan</code> (Hz)	7,86	0,60–1,25
Desviación típica del intervalo (s)	0,006–0,008	2,5–2,8
Intervalo máximo entre mensajes (s)	0,14	16,12
Tamaño de cada mensaje (KB)	8,71	8,71
Caudal aproximado (KB/s)	≈65	n. d.

Bajo carga, la frecuencia de entrega del escáner cayó de 7,86 a en torno a 0,6 Hz (un factor de trece), la desviación típica del intervalo entre mensajes se disparó de unos 7 ms a 2,8 s

(aproximadamente un factor de cuatrocientos) y se registraron huecos de hasta 16,12s entre escaneos consecutivos. Este último dato cierra el mecanismo causal: la caché de transformadas de Nav2 retiene 10s de historia, de modo que un escaneo que llega con 16s de retraso encuentra una caché cuyo dato más antiguo es *posterior* a su propia marca temporal y es descartado, que es exactamente el error observado de forma continua durante la sesión. Privado del láser, `slam_toolbox` construye el mapa a trompicones y `explore_lite` no logra completar los objetivos de navegación.

Conviene subrayar que **no se trata de una saturación de ancho de banda**: el escáner son mensajes de 8,71 KB, muy por debajo de la capacidad de la red. La degradación aparece únicamente con la pila completa en marcha, lo que apunta a una contención en la capa de transporte y virtualización del banco de pruebas (el puente wifi de la máquina virtual y el tráfico DDS con reintentos de la decena de nodos de Nav2) más que a un problema de la red inalámbrica en sí. Los datos son, por tanto, consistentes con un cuello de botella de transporte inducido por el entorno de pruebas, y no por el software del proyecto, cuya lógica de navegación se activó correctamente, ni por los relojes o la CPU, descartados por medición.

6.4.5 Detección de incidencias: no evaluada por un fallo de la cámara

La validación de `detect_incident` sobre el robot real no pudo completarse debido a que la cámara OAK-D-LITE no entregaba imágenes. El nodo en sí funcionó hasta donde el hardware lo permitió: arrancó correctamente, cargó el modelo YOLOv8n desde disco (sin acceso a Internet), se suscribió a la cámara con la política de calidad de servicio adecuada y estableció la conexión con MinIO y Kafka. Sin embargo, no llegó a recibir un solo fotograma que procesar, porque la cámara no publicaba datos en su tópico de imagen.

Al buscar el origen, el problema se localizó en el propio robot, y no en la red ni en el software del proyecto: el tópico de la cámara aparecía sin ningún publicador incluso al comprobarlo en la propia Raspberry Pi, sin enlace inalámbrico de por medio. El registro del sistema mostró que, en algunos arranques del servicio, el controlador de la cámara no llegaba a iniciarse. Los dos extractos siguientes contrastan un arranque correcto con uno defectuoso.

```
title=Registro del robot: arranque correcto de la camara (referencia)

[turtlebot4.oakd]: Camera with MXID: 18443010E10E5F0E00 and Name: 1.1.1
connected!
[turtlebot4.oakd]: USB SPEED: SUPER
[turtlebot4.oakd]: Device type: OAK-D-LITE
[turtlebot4.oakd]: Camera ready!
```

```
title=Registro del robot: arranque defectuoso (sin controlador de camara)

[turtlebot4_node]: OAKD started
[turtlebot4_node] [ERROR]: Service oakd/start_camera unavailable.

$ ros2 topic info /turtlebot4/oakd/rgb/preview/image_raw --verbose
Type: sensor_msgs/msg/Image
Publisher count: 0 # nadie publica: el controlador no se instancio
```

En el arranque defectuoso no aparece ninguna traza del controlador *DepthAI* (ni **Camera ready!** ni la conexión del dispositivo) y el recuento de publicadores del tópico de imagen es cero, pese a que el nodo del robot da la cámara por iniciada. Con la evidencia disponible, la hipótesis más plausible es que la imposibilidad de evaluar la detección en el robot real obedece a una indisponibilidad del sensor o a un fallo intermitente de la cámara o de su controlador en la Raspberry Pi, siendo este ajeno al software desarrollado ya que su funcionalidad quedó verificada en la simulación (Sección 6.3.2).

6.4.6 Síntesis de la campaña real

La tabla 6.6 resume el estado de validación de cada subsistema sobre el hardware real. En conjunto, la campaña confirma que la arquitectura del proyecto opera contra el robot físico (autenticación, telemetría, cadena de mando y activación de la navegación quedaron demostradas) y acota con precisión, y mediante medidas, los dos factores que impidieron cerrar el ciclo completo: la degradación del transporte en el banco de pruebas y el fallo del sensor de visión. Ambos son externos al software desarrollado, lo que refuerza la validez de la validación funcional obtenida en simulación (Sección 6.2) y delimita el trabajo de despliegue restante.

Tabla 6.6: Estado de validación de los subsistemas sobre el robot real. Fuente: elaboración propia.

Subsistema	Observación en hardware real	Resultado
Conectividad y descubrimiento	Doble servidor de descubrimiento; transformadas y tópicos del robot visibles	Validado
Sincronización temporal	Tres relojes sincronizados (<10 ms) tras aislar VirtualBox	Validado
Autenticación (secret_helper)	TOFU y JWT contra el <i>backend</i> real	Validado
Telemetría (serve_telemetry)	Robot → InfluxDB → Grafana y aplicación	Validado
Orquestación (FSM)	Orden por Kafka; transición a exploración; acople/desacople	Validado
<i>Bringup</i> de Nav2	Arranque completo hasta Nav2 is ready for use!	Validado
Cartografía y exploración	Mapa parcial; limitado por el jitter de transporte (tabla 6.5)	Parcial
Detección (detect_incident)	Nodo correcto; sensor no disponible (fallo de la cámara)	No evaluable

6.5 Discusión

Los resultados cuantitativos obtenidos en simulación, junto con la validación funcional, permiten afirmar que la arquitectura propuesta es coherente y que sus componentes se integran según el diseño previsto. El desacoplamiento mediante el bus de eventos ha facilitado el desarrollo y la prueba de cada subsistema por separado: cada flujo se ha podido ejercitar de forma aislada y, después, en conjunto, sin que la indisponibilidad puntual de un componente bloqueara a los demás.

La campaña sobre el robot real (Sección 6.4) refuerza esta lectura sobre hardware, y no solo en simulación. La autenticación, la telemetría, la cadena de mando del orquestador y la activación de Nav2 funcionaron contra el TurtleBot 4 físico sin modificar el software, y el desacoplamiento por eventos volvió a mostrar su valor: el fallo de la cámara no impidió que la telemetría, la autenticación y la navegación siguieran operando. Los dos factores que impidieron cerrar el ciclo completo, la degradación del transporte de datos bajo carga en el banco de pruebas y el fallo intermitente de la cámara, se aislaron, justificaron y cuantificaron, resultando ambos externos a la arquitectura propuesta. Este comportamiento respalda la decisión de diseño de articular el sistema en torno a un bus de eventos.

Conviene recordar que buena parte de esta validación procede de la simulación, que no reproduce con total fidelidad las condiciones del entorno real. Aun así, el comportamiento observado sobre el robot físico fue consistente con el previsto, y las incidencias surgidas durante la campaña se explicaron por el banco de pruebas y por la disponibilidad puntual del hardware, no por el sistema desarrollado. El balance es, por tanto, positivo: la arquitectura ha demostrado ser coherente, funcional y fiel a las decisiones de diseño que la sustentan.

7 Conclusiones

Este capítulo recapitula el trabajo realizado, valora el grado de consecución de los objetivos planteados, expone las aportaciones y limitaciones, y traza las líneas de trabajo futuro. Por último, se incluye una breve valoración personal.

7.1 Grado de consecución de los objetivos

El desarrollo de este Trabajo ha permitido construir un sistema robótico de patrulla autónoma funcional. Retomando los objetivos específicos formulados en el Capítulo 3, puede afirmarse que se ha implementado la cartografía autónoma del entorno mediante exploración y SLAM (OE1) y la generación y ejecución de una ruta de cobertura con Nav2 (OE2). El ciclo operativo se coordina mediante un orquestador en forma de máquina de estados que encadena exploración, mapeado, planificación, patrulla y retorno al punto de carga (OE3). Sobre esa base, se ha integrado la detección de personas como incidencia con YOLO (OE4), se ha diseñado la arquitectura orientada a eventos que desacopla el robot del backend (OE5) y se ha desarrollado el servicio de backend con persistencia de datos relacionales, de vídeo y de series temporales (OE6). Asimismo, se ha implementado el esquema de seguridad basado en TOFU, JWT y control de acceso por roles (OE7) y se ha construido la aplicación de usuario multiplataforma con Flutter (OE8). Por último, la integración de todos los subsistemas en una solución desplegable mediante contenedores se ha completado y validado de extremo a extremo en simulación (OE9).

Conviene diferenciar el nivel de evidencia que respalda cada afirmación. Todo el ciclo se ha validado de extremo a extremo en simulación, sobre la misma pila de software que ejecutaría el robot. Además, una parte del sistema se ejercitó sobre el TurtleBot 4 físico, donde la autenticación, la telemetría, la cadena de mando del orquestador y la activación de la navegación funcionaron sin modificar el software (Sección 6.4). Lo que no llegó a completarse fue una evaluación cuantitativa en condiciones reales, limitada por factores externos al proyecto y planteada como trabajo futuro.

7.2 Aportaciones

La principal aportación del trabajo es la **integración de extremo a extremo** de un sistema de vigilancia robótica con tecnologías de código abierto, que cubre desde la cartografía y la patrulla autónoma hasta la notificación de incidencias y su consulta desde una aplicación multiplataforma. Frente a soluciones centradas en un único subproblema, este trabajo articula navegación, percepción, mensajería, servicios web y aplicación de usuario en un único sistema coherente y desplegable. Más allá de esa integración, pueden destacarse varias aportaciones concretas:

- **Arquitectura orientada a eventos.** El desacoplamiento de la capa robótica y el backend mediante un bus de mensajería ha demostrado su valor al permitir desarrollar, probar y desplegar cada componente de forma independiente, y al absorber la indisponibilidad temporal de cualquiera de ellos sin pérdida de eventos.
- **Orquestación autónoma del ciclo de patrulla.** El diseño del orquestador como una máquina de estados que encadena exploración, mapeado, planificación y patrulla encapsula en un único nodo la coordinación de capacidades heterogéneas, y reutiliza el mapa cuando ya existe para no reexplorar el entorno innecesariamente.
- **Modelo de seguridad para el robot.** La autenticación del robot mediante el patrón TOFU y tokens JWT, con doble verificación de identidad (en las dependencias del backend y, de nuevo, en el consumidor de eventos), aporta un esquema de confianza adecuado a un dispositivo que opera sin supervisión continua.
- **Despliegue reproducible.** La descripción declarativa de la infraestructura mediante contenedores permite levantar el sistema completo de forma uniforme, lo que facilita tanto el desarrollo como una futura puesta en producción.

7.3 Limitaciones

El trabajo presenta varias limitaciones que conviene reconocer. La estrategia de planificación de la ruta de cobertura es deliberadamente sencilla (un barrido en zig-zag) y puede resultar subóptima en estancias de geometría compleja. La detección se limita actualmente a la presencia de personas, sin seguimiento de objetos ni distinción entre individuos, lo que puede generar alertas redundantes ante una misma persona. Además, la percepción depende de una única cámara frontal, de modo que el robot conserva ángulos ciegos a los lados y en la parte trasera durante la patrulla.

En el tratamiento de las incidencias subsiste otra limitación: el sistema se queda en detectar la presencia de una persona, sin clasificar ni describir el suceso. Para enriquecer ese tratamiento se ingenió e intentó integrar un Vision-Language Model (VLM) ligero, Qwen2.5-VL-3B-Instruct cuantizado a 4 bits, servido con vLLM a través de una API compatible con OpenAI, que clasificara y describiera en lenguaje natural cada incidencia a partir de su imagen. Sin embargo, no fue posible llevarlo a producción: el servicio de inferencia exige una GPU con memoria suficiente, y el hardware del servidor no podía alojarlo a la vez que el resto del sistema (backend, bases de datos, mensajería y visualización), mientras que ejecutarlo en el propio robot tampoco era viable por sus recursos limitados. La integración se retoma como línea de trabajo futuro.

En el plano de la seguridad deben reconocerse dos limitaciones conocidas: el patrón TOFU asume que el primer contacto del robot con el backend es legítimo, asunción que sería explotable si un atacante se adelantara al aprovisionamiento; y la política de CORS permanece abierta. Ambas resultan asumibles en el despliegue local y controlado actual, pero deben enducirse antes de un uso en producción. Por último, algunas capacidades que se contemplaron en un principio no forman parte de la implementación actual, en concreto la transmisión de vídeo en directo, el seguimiento multiobjeto y las notificaciones en tiempo real; todas ellas se recogen como líneas de trabajo futuro.

7.4 Trabajo futuro

A partir de las limitaciones anteriores se identifican las siguientes líneas de trabajo futuro:

- **Evaluación cuantitativa sobre hardware real.** Ejecutar una campaña de pruebas sistemática sobre el robot real que permita obtener las métricas previstas en el Capítulo 6 y recalibrar los parámetros ajustados en simulación.
- **Endurecimiento de la seguridad.** Restringir la política de CORS y reforzar el aprovisionamiento inicial del robot frente al riesgo del primer contacto, de cara a un despliegue en producción.
- **Seguimiento multiobjeto.** Incorporar un algoritmo de seguimiento que asigne identificadores persistentes a las personas detectadas y reduzca las alertas redundantes.
- **Transmisión de vídeo en directo.** Añadir la difusión del vídeo en tiempo real para permitir la supervisión remota durante la patrulla.
- **Notificaciones en tiempo real.** Avisar al usuario de forma inmediata ante una incidencia mediante notificaciones *push* o canales de comunicación en tiempo real.
- **Cobertura visual con cámara 360°.** Sustituir o complementar la cámara frontal por una cámara de 360° que elimine los ángulos ciegos del robot durante la patrulla.
- **Clasificación y descripción de incidencias con un VLM.** Retomar la integración del modelo de lenguaje y visión ligero (Qwen2.5-VL servido con vLLM) sobre hardware de inferencia dedicado o un servicio externo, para enriquecer cada incidencia con una etiqueta y una descripción en lenguaje natural, a partir del intento descrito en la Sección 7.3.

En relación con esta última línea, el extracto 7.1 recoge la configuración del servicio de inferencia con vLLM (`vllm/docker-compose.yml`) que se preparó durante el intento descrito en la Sección 7.3 y que constituye el punto de partida para retomarla.

Servicio de inferencia con vLLM (`vllm/docker-compose.yml`)

```
1 services:
2   vllm:
3     image: vllm/vllm-openai:latest
4     container_name: qwen25-vl-3b
5     runtime: nvidia
6     deploy:
7       resources:
8         reservations:
9           devices:
10            - driver: nvidia
11              count: all
12              capabilities: [gpu]
13     shm_size: "8gb"
14     volumes:
```

```
15     - ~/.cache/huggingface:/root/.cache/huggingface
16     ports:
17     - "8000:8000"
18     ipc: host
19     command: >
20     --model unsloth/Qwen2.5-VL-3B-Instruct-unsloth-bnb-4bit
21     --tokenizer Qwen/Qwen2.5-VL-3B-Instruct
22     --served-model-name qwen2.5-vl-3b
23     --quantization bitsandbytes
24     --load-format bitsandbytes
25     --dtype bfloat16
26     --max-model-len 32768
27     --gpu-memory-utilization 0.8
28     --max-num-seqs 4
29     --limit-mm-per-prompt '{"image":2}'
30     --trust-remote-code
31     restart: unless-stopped
```

- **Planificación de cobertura avanzada.** Sustituir la rejilla en zig-zag por estrategias de cobertura más eficientes y adaptadas a la geometría del entorno.
- **Operación multirobot.** Extender el sistema para coordinar varios robots patrullando un mismo espacio.

7.5 Valoración personal

La realización de este trabajo me ha permitido integrar en un único proyecto conocimientos de áreas muy diversas (robótica, visión por computador, sistemas distribuidos y desarrollo de aplicaciones) y enfrentarme a los retos que surgen al hacer que componentes heterogéneos colaboren de forma coherente. Construir el sistema de extremo a extremo, y no solo una de sus partes, ha sido la faceta más exigente y, a la vez, la más enriquecedora del trabajo.

Más allá del resultado técnico, considero que la principal lección ha sido la importancia de un buen diseño arquitectónico: las decisiones tomadas al principio, en especial el desacoplamiento mediante eventos, condicionaron favorablemente todo el desarrollo posterior y facilitaron cada incorporación sucesiva. Llevar el sistema hasta completar su validación sobre hardware real y su evaluación cuantitativa es el paso natural con el que me gustaría dar continuidad a lo aquí construido.

Bibliografía

- Alvarez, C. (2022). *m-explore-ros2: Lightweight Frontier-Based Exploration (ROS 2 Port)* [Port a ROS 2 de m-explore de Jiří Hörner]. Consultado el 24 de junio de 2026, desde <https://github.com/robo-friends/m-explore-ros2>
- Chiang, S.-Y., & Huang, S.-E. (2026). Design of a Patrol and Security Robot with Semantic Mapping and Obstacle Avoidance System Using RGB-D Camera and LiDAR. *Computers, Materials & Continua*, 87(1). <https://doi.org/10.32604/cmc.2025.074528>
- Di Paola, D., Milella, A., Cicirelli, G., & Distante, A. (2010). An Autonomous Mobile Robotic System for Surveillance of Indoor Environments. *International Journal of Advanced Robotic Systems*, 7(1). <https://doi.org/10.5772/7254>
- Durrant-Whyte, H., & Bailey, T. (2006). Simultaneous localization and mapping: part I. *IEEE Robotics & Automation Magazine*, 13(2), 99-110. <https://doi.org/10.1109/MRA.2006.1638022>
- Galceran, E., & Carreras, M. (2013). A survey on coverage path planning for robotics. *Robotics and Autonomous Systems*, 61(12), 1258-1276. <https://doi.org/10.1016/j.robot.2013.09.004>
- Islam, M., Ahmed, T., Nuruddin, A. T. B., Islam, M., & Siddique, S. (2020). Autonomous Intruder Detection Using a ROS-Based Multi-Robot System Equipped with 2D-LiDAR Sensors.
- Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLO* (Ver. 8.0.0). <https://github.com/ultralytics/ultralytics>
- Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: a Distributed Messaging System for Log Processing. *Proceedings of the NetDB Workshop*, 1-7.
- Macenski, S., & Jambrecic, I. (2021). SLAM Toolbox: SLAM for the dynamic world. *Journal of Open Source Software*, 6(61), 2783. <https://doi.org/10.21105/joss.02783>
- Macenski, S., Foote, T., Gerkey, B., Lalancette, C., & Woodall, W. (2022). Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66), eabm6074. <https://doi.org/10.1126/scirobotics.abm6074>
- Macenski, S., Martín, F., White, R., & Ginés Clavero, J. (2020). The Marathon 2: A Navigation System. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2718-2725. <https://doi.org/10.1109/IROS45743.2020.9341207>
- Quigley, M., Conley, K., Gerkey, B., Faust, J., Foote, T., Leibs, J., Wheeler, R., & Ng, A. Y. (2009). ROS: an open-source Robot Operating System. *ICRA Workshop on Open Source Software*, 3(3.2), 1-6.
- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 779-788. <https://doi.org/10.1109/CVPR.2016.91>

Lista de Acrónimos y Abreviaturas

API	Application Programming Interface. xiii, 2, 9, 11, 13–15, 20, 21, 34, 47
CNN	Convolutional Neural Network. 5
CORS	Cross-Origin Resource Sharing. 25, 56, 57
CPU	Central Processing Unit. 12, 46, 51, 52
DDS	Data Distribution Service. 3, 47, 52
FSM	Finite State Machine. 9, 19, 28, 41, 50, 53
GPU	Graphics Processing Unit. 5, 12, 20, 56
HTTP	HyperText Transfer Protocol. 21, 23
JSON	JavaScript Object Notation. 23
JWT	JSON Web Token. 10, 13, 15, 18, 19, 23–25, 31, 33, 34, 42, 48, 49, 53, 55, 56
LiDAR	Light Detection and Ranging. 4, 6, 12, 47, 48, 50
Nav2	ROS 2 Navigation Stack. xiii, 2, 4, 5, 7, 9, 12, 26, 29, 41, 42, 50–55
ORM	Object-Relational Mapping. 13, 15, 21
R-CNN	Region-based Convolutional Neural Network. 5
RBAC	Role-Based Access Control. 25
REST	Representational State Transfer. xiii, 2, 6, 9, 14, 15, 20, 21, 34
ROS 2	Robot Operating System 2. v, xiii, 1–4, 6, 7, 11, 12, 14, 15, 19, 20, 25, 28, 33, 41, 47
S3	Simple Storage Service. 13, 15
SDK	Software Development Kit. 13
SLAM	Simultaneous Localization and Mapping. xiii, 2, 4, 6, 7, 9, 12, 26, 28, 55
SQL	Structured Query Language. 13
TOFU	Trust On First Use. 10, 15, 19, 21, 24, 32, 33, 42, 48, 49, 53, 55, 56
TSDB	Time-Series Database. 13
URL	Uniform Resource Locator. 18, 19, 23, 25, 36
VLM	Vision-Language Model. 56, 57
YOLO	You Only Look Once. xiii, 2, 5–7, 9, 12, 15, 26, 30, 42, 55